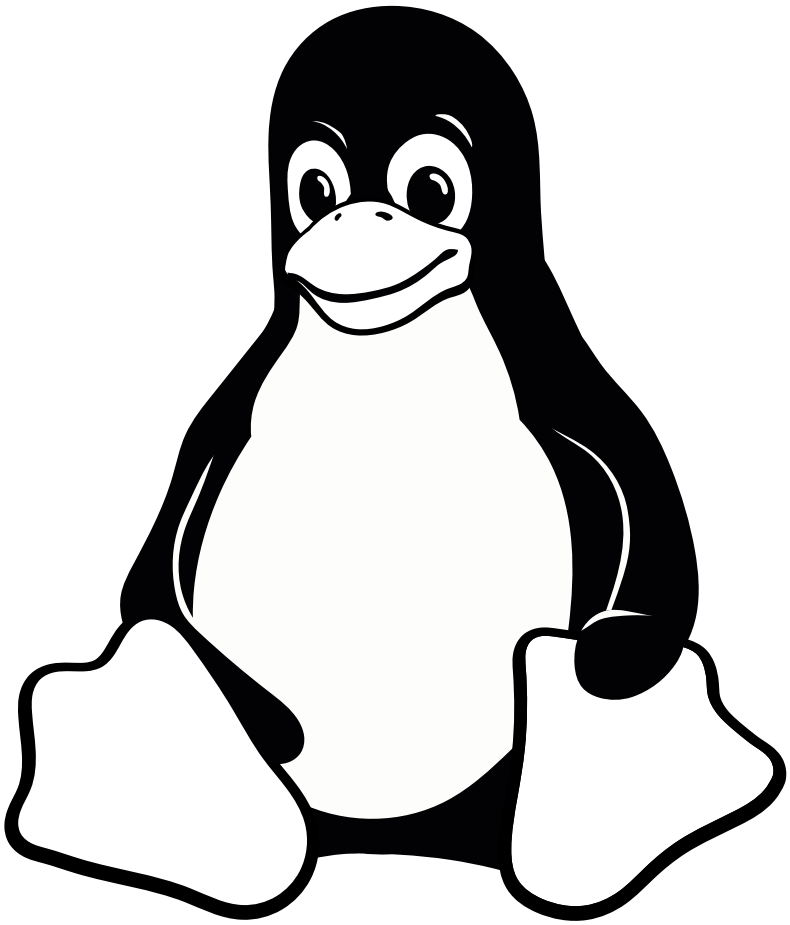
Pablo Rodríguez

# Aprender *Linux*

*Una introducción a la informática*



http://www.aprender-linux.tk

# *Aprender* Linux

## *Una introducción a la informática*

Pablo Rodríguez

2017

http://www.aprender-linux.tk

© 2017 Pablo Rodríguez (http://www.aprender-linux.tk). Todos los derechos reservados.

El Centro Español de Estudios Repográficos —CEDRO— carece de autorización para percibir derechos de ninguna naturaleza por este libro. Esta obra no figura en su catálogo.

No son objeto de copia privada «las efectuadas en establecimientos dedicados a la realización de reproducciones para el público, o que tengan a disposición del público los equipos, aparatos y materiales para su realización» (Real Decreto 1657/2012, artículo 3.4.a, https://www.boe.es/buscar/act.php?id=BOE-A-2012-14904#a3).

*A Daniel, por la excusa.*
*A Óscar, por la oportunidad.*

Existen 10 tipos de personas:
quienes saben de informática
y quienes escriben los libros[1].

# CONTENIDO

Sobre la licencia de uso 11
Prólogo 13
Introducción 15

¿QUÉ ES *LINUX*?

1 *Linux* y la informática 21
2 ¿Por qué *Linux*? 30
3 Una filosofía distinta 37
4 La programabilidad 46

RECONOCIENDO EL TERRRENO

5 En tierra extraña 57
6 El asistente digital 64
7 La comunicación real 74

Notas 83

# SOBRE LA LICENCIA DE USO

Este libro se ofrece con descarga gratuita en `http://www.aprender-linux.tk`. Esta obra no está bajo una licencia *Creative Commons* u otra licencia libre similar.

Mi intención es poner este libro a disposición de toda persona que quiera leerlo. Espero que sea útil. A cambio, sólo pido un sencillo pacto de nobleza. Creo que cualquiera estará de acuerdo.

- Si el libro te parece útil, recomiéndalo a quien le pueda interesar. Hazle llegar la dirección `http://www.aprender-linux.tk`. Así tendrá la última versión disponible. También sabré el número de descargas con bastante fiabilidad.

- Si consideras que tiene puntos mejorables, no dejes de decir cuáles son. Tampoco olvides de describir en qué deben mejorar.

- Si consideras que el libro es malo, agradezco la crítica. La única condición es que sea respetuosa con las personas.

Los comentarios son incidencias en `https://github.com/ousia/aprender-linux/issues/`. Tienes que estar inscrito en *GitHub*. Todos los comentarios son públicos.

Para que no haya incomprensiones, hago las aclaraciones pertinentes. Así nadie se lleva a engaño.

Únicamente se permite la descarga desde la página mencionada. Se permite el uso personal —no colectivo— de los archivos. Si quieres compartir el libro, envía `http://www.aprender-linux.tk`.

Dentro del uso personal, se incluye la impresión del archivo PDF en establecimiento público. En la página se explica cómo ha-

cerlo. El establecimiento no está autorizado a vender el libro, sólo a imprimirlo a petición de la persona interesada. El precio de impresión y encuadernado no podrá ser superior al del mismo número de páginas y encuadernación de cualquier otro documento.

Una última aclaración para las entidades de gestión colectiva de derechos —Centro Español de Derechos Reprográficos, CEDRO—: no cabe el cobro de licencia por fotocopia de esta obra. Este título no se encuentra dentro de su catálogo.

No existe copia privada en fotocopiadoras de uso público por excepción reglamentaria (Real Decreto 1657/2012, artículo 3.4.a, `https://www.boe.es/buscar/act.php?id=BOE-A-2012-14904#a3`).

Por tanto, no pueden cobrar por esta obra. Ni por copia privada, ni por cualquier acuerdo o programa de licencias con su entidad —`https://www.conlicencia.com`—. Ustedes carecen de ella.

# PRÓLOGO

Estas páginas surgen de una situación muy concreta. Un gran amigo mío tenía un ordenador relativamente antiguo y con pocos recursos. Me dijo que era una pena, porque aunque no había sido caro, no podía usarlo para nada. *Windows* iba muy lento y no podía hacer casi nada con él. De hecho, no lo usaba.

Con su generosidad habitual, me lo ofreció si yo sabía sacarle partido. Le dije que podría usarlo con un sistema operativo que necesitase menos recursos. Así iría más rápido y trabajaría mejor con él. Sólo quería usarlo para escribir y para escuchar música. No le oculté que era *Linux*.

La siguente vez que nos vimos —meses después—, me dediqué a instalarle una versión de *Linux* en el ultraportátil. Reiteró su generosa oferta de dármelo si me servía. Le contesté que le podría servir a él. De hecho, al verlo funcionar, se dio cuenta de que podría usarse para lo que quería. Y también para aprender a usar *Linux*.

Tras unos meses de uso sin problemas, traté de actualizarles la versión de *Linux*. Por falta de previsión y de tiempo —vivimos en ciudades distintas—, me tuve que llevar el ultraportátil a casa. Lo actualicé a la mañana siguiente. Lo llevo usando casi un mes y espero devolverlo en unas semanas.

Gracias al sistema adecuado, tengo que reconocer que es el ordenador prefecto para escribir este libro. Funciona mejor que el primer día. No tiene más potencia o recursos, aunque es bastante probable que gestione mejor los que ya tenía desde un principio.

Estas páginas son la explicación de qué es y cómo funciona *Linux*. Espero que puedan ser útiles a cualquiera que quiera introducirse en el uso de este sistema operativo. Creo que también

puede ser útil para entender y aprender sobre el funcionamiento básico de un ordenador.

20 de enero de 2017

# INTRODUCCIÓN

Antes de empezar a explicar, creo que es importante describir qué vas a encontrar cuando leas estas páginas. Es bueno para saber para qué y para quién pueden ser útiles.

En primer lugar, tengo que presentarme. Llevo usando *Linux* como único sistema operativo en mi ordenador desde hace más de una docena de años. No soy informático, ni tampoco sé programar. Mi formación es en humanidades —soy «de letras»—. En realidad, sólo soy un usuario medio de ordenadores. Aunque creo que sé cómo funcionan. Se verá en las páginas que siguen.

Lo que pretendo explicar aquí es cómo manejar con cierta soltura un ordenador al que se le instale *Linux*. No requiere conocimientos previos de este sistema operativo, pero sí exige haber usado ordenadores antes. Y tener cierta familiaridad de uso informático —aunque sea con *Windows*—. Al menos, haber usado con cierta frecuencia un procesador de textos, un programa de hoja de cálculo, y un navegador de internet. Será útil también haber comprimido algún archivo, así como haber instalado algún programa en el ordenador que usemos habitualmente[2].

¿Qué pasa si sabes más? Realmente nada. Es fácil que aprendas más rápido. O que necesites libros más específicos para aprender lo que necesitas a tu nivel. Quizá no sea buena idea pensar que ya sabes —o que no podrás aprender— por un punto concreto. Es bueno seguir leyendo para poder conseguir una conclusión mejor razonada.

Respecto a quien lea, sólo espero esfuerzo por entender. En realidad, se trata de dos cuestiones. Todo aprendizaje exige y supone esfuerzo. No es sufrimiento, sino sencillamente seguir queriendo aprender. Esto significa buscar aprender cuando las cosas se

ponen difíciles. Por mi parte, las explicaciones serán lo más claras posible. Pero yo puedo aprender sólo por mí, no por quienes lean. Cada cual tiene que aprender para sí. Eso supone esfuerzo.

En lo relativo a entender, hay algo quizá extraño en la enseñanza y el aprendizaje de la informática. Quizá lo veo más claramente porque no soy informático. Aprender informática es aprender cómo funcionan los ordenadores. Sólo si sabemos como procesan la información, podremos trabajar mejor con ellos. Eso supone entender cómo trabajan los ordenadores. Si entendemos, tendremos mucho aprendido, porque los ordenadores son máquinas. Si no entendemos, tendremos que memorizar que haciendo algo concreto —pulsando una tecla, o pinchando con el ratón en un punto— conseguimos un resultado. Eso se parece más a la magia. Los ordenadores son calculadoras. No hay que saber programarlas, pero hay que saber manejarlas. Porque para trabajar con máquinas lógicas —eso son los ordenadores—, o trabajan para ti, o acabas trabajando tú para ellas.

De ahí que el esfuerzo por entender sea decisivo. Puede llegar a costar un poco más al principio. Significa no contentarnos con cómo se hace algo en el ordenador, sino saber qué está haciendo. Por si no es claro, lo explico de otra forma. Saber no es repetir lo que nos dicen otros, sino hacer que entienda quien no sabe nada de lo que le contamos. Si no sabemos explicar a quien no sabe, no hemos entendido nada. Aunque tengamos el convencimiento de lo contrario. Por eso es necesario un esfuerzo.

Ese esfuerzo por entender es una inversión en aprendizaje. El gasto inicial de tiempo —y neuronas— lo recuperas con un aprendizaje posterior más rápido y mejor. Como entiendes, no necesitas tantas explicaciones. Lo que sabes también lo sabes mucho mejor. No te entran dudas, porque no sólo conoces, sino que realmente sabes.

La única manera de llegar a saber cuando ignoramos es el aprendizaje. Nos pasa a todos con todo. Si quieres, déjame que te explique *Linux* a lo largo de estas páginas que siguen. Y conseguirás aprender y entender cómo funciona un ordenador.

# ¿QUÉ ES *LINUX*?

# 1 *LINUX* Y LA INFORMÁTICA

*A El sistema operativo B Una gran calculadora C El núcleo*
*D El funcionamiento del núcleo E Los periféricos*

*Linux* es un sistema operativo de programación informática libre. Es también una marca registrada en muchos países a nombre de su creador, Linus Torvalds. Su desarrollador principal es informático finés, nacionalizado estadounidense en 2010. Comenzó en la década de los noventa a escribir el código de *Linux*. Después de conseguir un gran éxito, su tarea fundamental ahora es incorporar al código las mejoras y correcciones que hacen otras personas.

Esta parte pretende explicar qué significa la primera frase del párrafo anterior. No se trata de dar detalles técnicos, sino de entender dónde está situado *Linux*. Así podrás saber qué puedes esperar de él.

## A EL SISTEMA OPERATIVO

Un ordenador funciona por dos elementos básicos:

- La parte física —en inglés, *hardware*— que es lo que se puede tocar, el dispositivo informático apagado.
- La parte lógica —en inglés, *software*— que son lo que llamamos los programas, con los que podemos hacer cosas cuando encendemos el ordenador.

El sistema operativo es la parte lógica de la informática. Necesita de un ordenador —de un dispositivo— para funcionar. Cuando encendemos el ordenador, lo que se ejecuta —antes que cualquier

programa— es el sistema operativo. Es el conjunto de programas que permiten que las personas podamos usar cualquier ordenador. Trabajar con el ordenador significa que las personas podamos usar programas.

Del párrafo anterior, puede surgirte la siguiente pregunta: si un sistema operativo es un conjunto de programas, ¿cuál es la diferencia con los programas que usamos las personas? Los programas del sistema operativo son programas básicos. Algunos los podemos usar las personas, pero otros muchos los necesita el propio ordenador para su funcionamiento.

Aunque luego te explicaré la parte física del ordenador, quiero que entiendas algo antes. Un ordenador no funciona sin electricidad y nunca recuerda nada. Es capaz de dejar guardadas cosas, pero siempre tiene que empezar de nuevo cada vez que se enciende. Cada vez que se enciende tiene que establecer un orden mínimo. Todo antes de poder empezar a trabajar, de poder responder a las peticiones de quien lo use. Todo antes de poder operar.

Imagínate si llegases a una casa completamente cerrada y tuvieses que encender la luz para ver dónde está todo. Antes de pudieses hacer nada relevante allí, tendrías que situarte: ver dónde está todo lo que vas a necesitar y de qué útiles vas a disponer. Si has estado antes, tendrás lo que ya has hecho otros días. Tu diferencia con el ordenador es que éste no recuerda nada. Aunque pueda recuperar lo que ha hecho y guardado antes. Un ordenador puede ser muy rápido, pero tiene que hacer esa «preparación» antes de estar listo.

La preparación anterior es la carga del sistema operativo. El ordenador —la parte física que veremos luego— aprende cada vez que lo encendemos qué puede hacer y cómo puede hacerlo. Si quieres, se lee el diccionario del idioma con el que va a poder realizar su tarea. No es otra cosa que hacer operaciones lógicas.

Aunque para saber cómo hacerlas, el ordenador no sólo necesita electricidad, sino unas instrucciones básicas para entender el resto de instrucciones.

Como veremos luego, un ordenador es tu asistente digital. Un robot que está a lo que le ordenes y mandes. Aprender informática es sólo saber cómo mandar a esa máquina enteramente a tu disposición. Para entenderte, tiene que leer cada vez que lo enciendes el diccionario de órdenes básicas. Sin eso, un ordenador no funciona. Esas órdenes básicas son el sistema operativo. Es el gran programa —o un sistema de programas— que permite que el resto de programas funcionen.

El sistema operativo que te voy a explicar es *Linux*. Es posible que te suene más *Windows*, que es otro sistema operativo de *Microsoft*. *MacOS X* es un sistema operativo de *Apple*, que es el fabricante de dispositivos electrónicos —ordenadores, tabletas, teléfonos «inteligentes», e incluso relojes «inteligentes»—. *Unix* es otro sistema operativo más antiguo, en el que se basan tanto *MacOS X*, como *Linux*. Por supuesto, existen más sistemas operativos, pero sólo quiero mostrarte los ejemplos más conocidos.

Todos los sistemas operativos hacen lo mismo: permitir que un ordenador funcione, que pueda usarse. Por supuesto, no todos lo hacen de la misma manera. Son parecidos, pero no iguales. Por eso, los programas para un sistema operativo no sirven en otro. Los programas de *Android* no sirven para *iOS*. O los programas de *Linux* no sirven para *Windows*. Cada sistema operativo necesita lo que se llaman programas nativos. Son programas propios, porque el diccionario inicial no les permite entender lo que está escrito en otro idioma.

## B UNA GRAN CALCULADORA

Un ordenador es una gran calculadora binaria. Sólo son capaces de gestionar físicamente unos y ceros. En realidad, el uno y

cero es un interruptor: encendido o apagado. O como una llave de paso: abierto o cerrado. Ahora, un ordenador son los millones de posibilidades que se dan con esos interruptores. Aunque no te preocupes, no te voy a explicar la parte electrónica de la informática. Yo mismo no la conozco.

Lo importante es entender la estructura física de un ordenador, porque así tendrás una idea de cómo funciona. Un ordenador es una máquina universal. ¿Qué quiere decir eso? Que puede hacer lo que sea siempre que no sea contradictorio. La diferencia de un ordenador con una calculadora es que la calculadora sólo puede hacer un tipo de operaciones. Un ordenador es una máquina lógica universal, porque puede hacer cualquier tipo de operaciones lógicas.

Dentro de un ordenador podemos distinguir el núcleo y los periféricos. La explicación siguiente es para que la entiendas, aunque yo mismo podría hacerle muchas matizaciones a muchos detalles. Lo importante no es saber si cada cosa es exactamente como la describo, sino si te permite tener una idea clara de qué convierte a un ordenador en una máquina universal.

Hay una advertencia importante para todo este libro. Cuando hablo de ordenador, me refiero a lo que es capaz de procesar informáticamente. Las explicaciones más prácticas serán útiles para ordenadores o portátiles de diferentes tamaños y formas. Sin embargo, un ordenador es también una tableta, un teléfono o reloj «inteligente» y muchos otros dispositivos que procesan datos. Todos tienen un sistema operativo o no funcionarían en absoluto. En muchos casos, es *Linux*.

### C EL NÚCLEO

Volvemos a la estructura del ordenador. El núcleo es la parte del ordenador que realiza operaciones lógicas. Dentro del núcleo

del ordenador, hay que distinguir tres componentes: el procesador, la memoria de trabajo y la memoria de almacenamiento. Esos tres componentes del núcleo suelen estar juntos. Pero forman parte del núcleo por las funciones que tienen, no por dónde están situados.

El procesador es el cerebro del ordenador. Es la parte que realiza los cálculos. Sólo puede hacer cálculos extremadamente sencillos. Aunque es capaz de realizar millones por segundo. De un procesador, lo más importante es la velocidad. Esta mide cuántas operaciones simples puede hacer un procesador por segundo[3]. La velocidad se mide a día de hoy en gigahercios. Un gigahercio es la capacidad de realizar mil millones de operaciones simples por segundo.

De un modo más o menos impreciso, podemos decir que el procesador es la parte que piensa del ordenador. De ahí que le llamen también el cerebro informático. En realidad, sólo calcula. Lo importante —olvida lo demás sin problemas— es que hace operaciones absolutamente simples a velocidades de auténtico vértigo.

Para que el núcleo del ordenador pueda hacer operaciones, es necesaria una memoria de trabajo. El procesador sólo sabe calcular lo que tiene delante. No tiene dónde almacenar lo que ha hecho ya. Sobre todo, no sabe lo que viene después. Sin memoria de almacenamiento, el procesador no sabría gestionar los datos con los que tiene que operar.

La memoria de trabajo se conoce en inglés como *random access memory* —memoria de acceso aleatorio—. Y en español, la conocemos por las siglas RAM. Es una memoria que tiene que ser rápida, para poder trabajar con el procesador. Pero no almacena ningún dato de forma permanente. Cuando el ordenador se apaga o pierde la corriente, los datos se borran de la memoria de almacenamiento. Es volátil.

No olvides nunca que un ordenador no recuerda. Sólo puede almacenar lo que ya ha trabajado. Por eso, necesita memoria de almacenamiento. Es más lenta que la de trabajo, pero hay mucha más memoria de almacenamiento que de trabajo. En la memoria de almacenamiento se leen datos y se escriben datos. La memoria de almacenamiento es lo que llamamos el disco duro, que no pierde datos cuando se apaga correctamente el ordenador.

Del disco duro, el procesador lee los datos cuando se enciende el ordenador. ¿Qué datos? Primero, los del sistema operativo para que funcione el ordenador. Después, lee también lee datos de los programas instalados para que puedas usarlos. Y de los documentos con los que quieras trabajar. Graba datos, tanto de lo que el ordenador haya hecho, como de lo que tú hayas hecho con él.

Sin uno de los tres elementos —procesador, memoria de trabajo y memoria de almacenamiento—, un ordenador no funciona. Sin procesador, el ordenador no trabajaría nada. Sin memoria de trabajo, el procesador estaría ciego y la memoria de almacenamiento no podría proporcionar ni recibir datos del procesador. Sin la memoria de almacenamiento podría llegar a funcionar un ordenador que nunca se apagase[4]. Al menos, esta última posibilidad es muy poco práctico.

## D EL FUNCIONAMIENTO DEL NÚCLEO

Un ordenador es principalmente su núcleo. No es sólo su núcleo, pero es una parte esencial. Es una parte necesaria y la más importante. Es la manera en que un ordenador trabaja. Tienes que entender que el ordenador trabaja con los tres elementos. Con sólo uno, sería imposible que trabajase. No podría procesar información, ni podríamos usarlo.

Entiendo que hay dos maneras de explicar cómo funciona un ordenador. No son explicaciones técnicas, sino métaforas que nos

permiten entender por dónde van los tiros. La primera imagen es la mano. La segunda es la escritura de un libro. Creo que ambos ejemplos dan una buena idea de cuál es el funcionamiento de un ordenador.

Imagínate una mano. No, mejor pon una mano abierta a casi medio metro de distancia de tu cara, mirando la palma. Mueve los dedos y obsérvalos. Gira la mano y considera que es una máquina. Es un instrumento de trabajo. La tarea más fácil sería recoger fruta en un cesto. Da igual de dónde cogieras la fruta. Lo importante es tenerla en el cesto. La velocidad de la mano es distinta de su tamaño. El cesto tiene un tamaño mayor que el de la mano, pero también está limitado.

Con una mano muy pequeña —como la de un bebé—, no podrías coger frutas grandes[5]. Sin cesto, no recogerías nada. Con uno muy pequeño, no recogerías apenas nada. Con una mano muy lenta, la recogida sería eterna. Tiene que haber una proporción —al menos— entre el tamaño y la velocidad de la mano. Porque una característica puede impedir la otra. Y el cesto ha de tener buena forma para que sea fácil dejar allí la fruta.

La segunda metáfora es la de la redacción de un libro. Para que el ejemplo sirva, pongamos que es un libro de citas. Para escribir un libro así tienes que escribir dos veces: en un cuaderno de notas y en las hojas en que será el libro final. Tienes que poder tomar notas muy rápido, para que ninguna nota se te pase. El cuaderno de notas tiene que ser el adecuado. Muy práctico y manejable, que puedas escribir muy rápido en él y que luego encuentres también las cosas rápido. No puede ser muy grande, para que sea tambíen cómodo. Y después de tomar todas esas notas, tendrías que redactar el libro. Ahí es importante que puedas escribir bien y que puedas poner todo por escrito para siempre.

La imagen de la mano muestra que la tarea de un ordenador la realiza el procesador. De un modo bastante complejo, hace cálculos a una grandísima velocidad. Aunque el procesador por sí solo no serviría para nada. Por muy rápido que sea, es importante que la memoria de trabajo —la memoria RAM— sea suficiente. Porque es quien gestiona los datos del procesador. Si es excesiva, también será un problema. Aunque en la práctica, es más habitual que falte memoria de trabajo y no que sobre[6].

La imagen de la escritura del libro indica que es importante también el acceso a la memoria de almacenamiento. Igual que con la memoria de trabajo, en la memoria de almacenamiento se lee y se escribe. Cuanto más rápido pueda ser ese intercambio, más rápido irán todos los procesos del ordenador. Con los ordenadores relativamente nuevos, sólo cambiando el disco duro normal por uno de estado sólido, se consigue un aumento muy importante en la velocidad.

En todo proceso con diferentes partes existe lo que se llama un cuello de botella. La cantidad de caudal en todo el recorrido es la del punto donde el conducto sea más estrecho. En un ordenador, la velocidad de proceso de datos será la del punto más lento.

Si lo quieres ver así, lo que te describo como el núcleo del ordenador —procesador, memoria de trabajo y memoria de almacenamiento— se puede comprender también como una silla. Necesitas las tres patas para mantener el equilibrio. Y siempre tendrás equilibrio hasta la altura de la pata más corta de las tres. De nada sirve que una parte —normalmente el procesador— sea extraordinariamente rápida. Porque tendrá que esperar a que el resto termine su trabajo.

### E LOS PERIFÉRICOS

Periférico es lo que está alrededor. ¿Qué es periférico del núcleo de un ordenador? Todo lo demás. O casi todo. Se me ocurre

una excepción, para que no te líes. La electricidad es parte esencial de la informática. Sin embargo, ni un enchufe, ni una batería son periféricos informáticos de un ordenador. Creo que una definición útil sería: periférico es aquel dispositivo que permite la entrada o salida de datos en el núcleo del ordenador.

¿Cuáles son los periféricos más comunes? Pues yo diría que pantalla, teclado y ratón. La pantalla —excepto que sea táctil— muestra datos. El teclado y el ratón —aunque sea lo que en inglés se llama *touchpad*— sólo introducen datos. Son periféricos las memorias secundarias de almacenamiento —se conecten como se conecten—, las cámaras controladas por el ordenador —sí, en inglés, *webcams*—. Por supuesto, también las impresoras de todo tipo, los digitalizadores de imagen —en inglés, *scanners*—. El resto sería muy largo.

Lo importante no es que tengas una lista infinita de periféricos. Lo relevante es que entiendas que todo lo que se conecta —de un modo u otro— al núcleo e intercambian datos, es un periférico. Como veremos más tarde, para que el ordenador pueda comunicarse con el periférico, es necesario un controlador. Así podrá entender los datos que recibe del periférico, y a su vez enviarle datos.

# 2 ¿POR QUÉ *LINUX*?

*A Un sistema operativo distinto* *B La utilidad de una impresora*
*C La propiedad inmaterial* *D Una protección excesiva*

## A UN SISTEMA OPERATIVO DISTINTO

En puridad, *Linux* no es un sistema operativo completo. Más bien, es el núcleo —*kernel* en inglés— de un sistema operativo. Para lo que usas —la última versión de *Fedora LXDE Edition*—, faltarían muchas cosas. Entre otras, un entorno gráfico de ventanas, que es lo que hace que te resulte parecido a *Windows* o a otros sistemas operativos móviles —como *Android* o *iOS*—. El sistema operativo montado con *Linux* tiene muchas más cosas que el núcleo. Ése es el motivo de que haya gente que prefiere llamar *GNU/Linux* al sistema operativo completo.

## B LA UTILIDAD DE UNA IMPRESORA

La historia tiene que ver con el proyecto *GNU* de Richard Stallman y la *Free Software Foundation*. Su intención era desarrollar un sistema *Unix* libre, con un modelo que te explicaré ahora. Todo comienza con una anécdota[7]. En 1980, Stallman trabajaba en el Laboratorio de Inteligencia Artificial del *Massachusetts Institute of Technology* —o *MIT*—. Tenían una impresora nueva —una *Xerox 9700*, donada por el fabricante— y Stallman quería el código que la manejaba —con el que se había escrito el programa controlador— para modificarlo. Ya lo había hecho con la impresora anterior.

Stallman necesitaba comunicar a quienes usaban la impresora, cuándo terminaban las impresiones o cuándo había atascos de

impresión. La mayoría no tenían la impresora a la vista, incluso estaban en distintos pisos. La impresora no traía el código fuente para añadir las modificaciones necesarias. Stallman no lo pudo conseguir, ni de *Xerox*, ni de nadie. Era un secreto comercial que impedía sacarle mayor provecho a la impresora. No prentendía fabricar impresoras más baratas y venderlas en el mercado negro. Sólo quería evitar gastar tiempo en esperar, ir y venir a una impresora común a todo un laboratorio universitario de investigación.

De esa experiencia, Stallman concluyó que la gente debía tener acceso al código fuente para modificarlo según le conviniese. Como regla general, las cosas son más útiles cuando las puedes adaptar a tus propias necesidades. Porque además, la utilidad no es general, siempre depende de las necesidades de quien usa las cosas. Sin el código fuente, es imposible mejorar los dispositivos informáticos o adaptarlos a nuestras necesidades. No hace falta saber programar, lo pueden hacer otras personas por ti.

De esa impresora salió el proyecto *GNU*. Es un proyecto de un sistema operativo libre tomando de que es el acrónimo recursivo de *GNU's Not Unix*. Trata de ofrecer un sistema completo en que la libertad sea total para modificar y distribuir el código. El único requisito es que las modificaciones tienen que ser públicas y debes otorgar al resto la misma libertad de la que has disfrutado. El proyecto *GNU* creó una licencia, la *General Public License*, en 1989. Esa licencia asegura que el código es libre para que cualquiera lo use, lo dé a otras personas o lo modifique. Esa licencia es la que tomará *Linux*.

## C LA PROPIEDAD INMATERIAL

Hay algo que quiero que entiendas. Probablemente tenga que hacer esta aclaración porque el modelo de desarrollo que tiene *Linux* ha tenido éxito. Otro ejemplo parecido de éxito es la *Wikipedia*. Uno de los problemas que tiene esta enciclopedia es ser una de las

diez direcciones más visitadas del mundo. Es la única de las diez que no vende nada, que no tiene ánimo de lucro. Su problema está en que tiene que tener una infraestructura muy grande para poder atender a tantas visitas. Sólo se financia con donaciones, no vende servicios, ni tiene siquiera publicidad. Las licencias *Creative Commons* serían otro ejemplo de una constante. Su formula sería que la protección legal por derechos de autor es excesiva.

Siento que el asunto sea un poco engorroso, pero antes de empezar es importante una aclaración. «Propiedad intelectual» en España es única y exclusivamente lo que está sometido a derechos de autor. Sí, lo que en el derecho británico y estadounidense se llama *copyright* —no existe en el resto del mundo, se llama derecho de autor—. En inglés, *intellectual property* es un término que sirve para nombrar tanto derechos de autor, como patentes, como marcas comerciales, como diseños protegidos. En España, ese campo sería propiedad innmaterial. Aunque realmente son derechos muy distintos, como ahora veremos. Por desgracia, traducir *intellectual property* como «propiedad intelectual» es una equivocación demasiado común[8].

Para explicar por qué la protección es excesiva, tengo que empezar con una frase un poco ardua. La propiedad inmaterial es la concesión de un monopolio temporal a cambio de un logro para el resto de la sociedad[9]. Por supuesto, ahora te explico la frase anterior. Los casos más claros de propiedad inmaterial son los derechos de autor y las patentes. Un ejemplo de derecho de autor es un libro, cuyo autor decide si se publica y quién lo puede publicar. Ejemplos de patentes pueden ser un medicamento que cure cualquier enfermedad o un invento que permita que las personas volemos individualmente —por pedir que no sea—.

La protección de derechos de autor y de patentes permite que quienes han creado o inventado algo tengan un período de tiempo en que esas personas deciden qué se puede hacer con sus obras o

inventos. ¿Es por el esfuerzo realizado? Realmente no. La exclusividad se otorga por su aportación a la sociedad. Por la nueva obra intelectual o el nuevo invento del que la sociedad se beneficia a medio o largo plazo. Porque después de veinte años, las patentes caducan. Tras ese plazo, cualquier persona puede usar y vender el invento. Y tras setenta años después de la muerte del autor, las obras pasan al dominio público.

Si la protección se otorgase sólo por el esfuerzo, protegeríamos la agricultura. No hablo en broma y te voy a explicar por qué la agricultura sería más digna de protección por esfuerzo que los derechos de autor. Cultivar exige mucho tiempo y esfuerzo. Además, depende de la climatología, que puede arruinar cualquier cosecha. El esfuerzo de la agricultura no está en la competencia por las ventas en un mercado libre. Por desgracia, a quienes se dedican a la agricultura, nada les puede asegurar que puedan obtener un producto para vender en ese mercado. Por algo tan sencillo como que dependen del cielo —el tiempo— y la tierra —de todo tipo de plagas y enfermedades—.

El monopolio es la exclusividad en la venta de una determinada mercancía. No hay monopolio cuando sólo yo puedo vender mis mercancías, como manzanas. El monopolio sería que sólo yo venda manzanas y todo el mundo me las tenga que comprar a mí. El problema del monopolio no es que alguien tenga todo, sino que sólo hay lo que tiene esa persona. Con el ejemplo de las manzanas, es muy grave que no pueda conseguirlas de otro vendedor. Aunque lo más grave es que ese vendedor controle todo lo que se puede hacer y vender con manzanas. Si hay monopolio de manzanas, quien las venda puede considerar que la tarta de manzana afecta negativamente a sus ventas. Independientemente de que afecte o no, podrías quizá hacer tarta de manzana en tu casa. Pero no podrías comprarla en una pastelería.

Una economía que quiera generar riqueza debe evitar los monopolios. En Estados Unidos y en la Unión Europea, existen normas que los prohíben. La única excepción son los derechos de propiedad inmaterial. No es porque sean un derecho fundamental[10], sino porque son una concesión para favorecer la aparición de novedades en el campo de la creación intelectual y de la invención técnica. Esa concesión siempre es temporal. Dure más o menos, la concesión siempre se otorga por un plazo determinado.

¿Todos sistemas de propiedad inmaterial son malos? No lo son, siempre que estén equilibrados. Sin ningún tipo de protección es fácil que haya pocos incentivos para dedicarse a ciertas actividades. Entre otras, a investigar o a crear obras. Pero si la protección es excesiva, lo que conseguiremos es que unos pocos controlen todas las novedades que vamos a tener. Lo peligroso del monopolio no es que todo esté en manos de uno —o unos pocos—, sino que no haya más que eso.

En el ámbito de la propiedad inmaterial, hasta el Vaticano ha advertido del exceso de protección. No lo cito como autoridad, sino para que veas que es algo que parece bastante obvio. Menos para quienes hacen las leyes —no siempre se puede tener todo en esta vida—. La regulación de derechos de autor y patentes es útil si sirve para el fin que persigue. Si no está bien hecha, destruye lo que realmente debe proteger.

### D UNA PROTECCIÓN EXCESIVA

En el caso de la programación, la protección en los últimos cincuenta años ha pasado de ser inexistente a ser excesiva. Los programas de ordenador al principio se daban con el código fuente. ¿Por qué? Porque lo importante era vender ordenadores, que no eran precisamente personales. Así, los programas se desarrollaban entre muchas más personas que sus autores originales. Había

mejoras y correcciones de fallos más rápido. Hay un momento en que eso cambia, con dos puntos.

El primer paso es afirmar que los programas de ordenador están protegidos por derechos de autor. Igual que si publicas un libro tienes derecho a decidir quién puede venderlo y a que te de parte de sus ganancias, los programas pasan a tener el mismo régimen. La segunda fase de protección de los programas de ordenador es que el código fuente se protege como un secreto comercial. Por tanto, te venden el programa final, no el código para que lo puedas adaptar. ¿No lo necesitas? Aunque no sepas programar, no sabes si te hará falta.

Derechos de autor y secretos comerciales son protecciones contradictorias si se aplican a lo mismo. Los derechos de autor protegen la publicación de este libro. ¿Por qué? Porque lo hago accesible al resto del mundo y así la gente puede aprender. El código del libro es su texto, el texto se publica y eso se protege. Con un programa de ordenador, la cosa cambia. El programa no es su código, el programa se protege con derechos de autor y el código también con secreto comercial. No es por nada, pero para decirlo con suavidad: no parece muy congruente.

Para liar más las protecciones de la informática, se añade una tercera: las patentes de programación. Como si fuesen inventos, los formatos de sonido o vídeo pueden estar patentados. De hecho, varios formatos lo están. No quiero discutirlo aquí, pero son patentes formales. Según la protección industrial a la que las patentes se refieren, las patentes formales carecen de sentido. Y las patentes de programación no tienen objeto. Aunque por su complejidad, es mejor que sólo sepas que existen patentes para programas de ordenador.

Volvemos a la contradicción entre derechos de autor y secretos comerciales para programas informáticos. No es congruente

proteger con derechos de autor un programa de ordenador. Puedes contestarme que si precisamente el *copyright* tiene que ver con el derecho de copia, lo que los derechos de autor regularían es la copia de programas informáticos. Antes de nada, la primera regulación de *copyright* del mundo es el Decreto de la Reina Ana aprobado en 1710 por el Parlamento británico. La copia se refiere a la impresión, a la publicación. Ésa era la posibilidad técnica en ese tiempo. El decreto no regulaba las copias excepto en el sentido descrito. Tiene tanto sentido proteger con derechos de autor un programa de ordenador, como un martillo o cualquier otro utensilio. Tendría sentido proteger bajo derechos de autor el código fuente. Pero aunque se protege y la protección es por su publicación, el código fuente está bajo secreto comercial.

Esa contradicción en la protección del código fuente es una protección excesiva o un sistema de protección totalmente desequilibrado. Un sistema de protección adecuado dentro del máximo que marca la ley sería proteger toda la informática con derechos de autor. Programa informático y código fuente no podrían copiarse ni publicarse, como sucede ahora. Pero la diferencia fundamental es que sería obligatorio otorgar el código fuente con el programa. Así, quien lo use, sabe qué hace el programa. Podría adaptarlo a sus necesidades. O podría conseguir que el programa comprado interactúe con otros programas.

Mi propuesta anterior es respecto a la protección de la informática que deberían ofrecer los derechos de autor. Todas las leyes de derechos de autor marcan un máximo. Por ejemplo, otorgan un plazo de setenta años de protección después de la muerte. Una persona puede decidir que sean menos, pero nunca pueden ser más. La propuesta del párrafo anterior es también un máximo. La uso para mostrar que incluso dentro de sistema de máximos, la protección es excesiva. Es totalmente distinta del modelo alternativo de desarrollo en que se basa *Linux*.

# 3 UNA FILOSOFÍA DISTINTA

*A Un modelo más eficaz de desarrollo*
*B Un modelo de cooperación y de negocio* *C Los recursos comunes*
*D Los tres sectores de la informática*

## A UN MODELO MÁS EFICAZ DE DESARROLLO

Como te acabo de explicar, la informática está protegida en Estados Unidos por el *copyright*. El modelo de desarrollo que tiene el proyecto *GNU* es la *GNU General Public License*. Esa licencia establece el marco legal para que el código y los programas de ese proyecto puedan desarrollarse del modo más eficaz posible. De esa licencia, existen tres versiones. Al igual que con los programas, la versión posterior mejora a la posterior[11]. *Linux* adopta la segunda versión de licencia para su desarrollo. También otros muchos proyectos están bajo esa licencia[12]. Vamos a ver en qué consiste ese nuevo modelo.

En primer lugar, si la protección legal habitual es el *copyright*, la *GNU GPL* ofrece lo que informalmente se conoce como *copyleft*. El *copyleft* obliga a que se mantengan los mismos derechos para el resto en todas las modificaciones del código y del programa. La licencia ofrece cuatro libertades básicas[13]:

1. Libertad para ejecutar el programa de la manera que quieras y mejor te convenga.

2. Libertad para estudiar el código y para modificarlo según quieras y te sea más útil.

3. Libertad para copiar el programa y así ayudar a la gente.

4. Libertad para poder publicar el programa modificado y que el resto de la sociedad puede beneficiarse de las mejoras.

La libertad básica es la utilidad del programa. El programa es útil para quien lo use, para adaptarlo a sus necesidades. Las otras tres libertades desarrollan esa libertad básica. Para el resto de libertades y para desarrollar la libertad básica es necesario tener acceso al código fuente del programa.

Vamos a verlo desde otra perspectiva. Es un ejemplo de la explicación del propio Stallman. Si vas al colegio con una bolsa con caramelos, todos los adultos te dirían que los compartieses el resto de la clase. A diferencia de los programas, los caramelos se acaban. Después del último, ya no hay más. La informática no tiene ese problema. Permite la reduplicación de archivos de ordenador sin apenas coste. Todas las copias tienen la misma calidad que el original. ¿Alguien puede dar más por menos?

Aplícalo a la informática. A este modelo de desarrollo se le llama en inglés *free software*. Por favor, no uses nunca «*software* libre», aunque sea muy común. No son, ni se trata de «programas gratuitos». En inglés, se da la siguiente explicación: *free as in free speech, not as in free beer*. Como sabes mejor que yo, se podría traducir como «libertad como en libertad de expresión, no como en barra libre». Por eso, *free software* es «programación libre». En realidad es un modelo de desarrollo, no un programa.

Con la programación —informática— libre puede que te vendan el programa. Lo más habitual es que no lo hagan. En todo caso, te tienen que dar acceso al código fuente. Puedes copiar para quien quieras el programa y el código, modificado o no. Puedes cobrar por eso o no. En todo caso, quien te ha proporcionado el programa no te puede impedir vender o dar el programa o el código a otras personas.

Supongo que el último párrafo te chocará. Aunque pueda ser relativamente frecuente la copia de programas —o sistemas operativos enteros— sin la autorización del titular de los derechos, siempre está la sombra de la pregunta. El interrogante no es otro que cómo se puede ganar dinero si no es vendiendo programas. La respuesta es cambiando el modelo económico. Te explico cómo.

La mayoría de programas que se escriben en el mundo no se publican para vender. De hecho, sólo una mínima proporción de programas que se escriben se publican. Porque en la gran mayoría de casos, los programas son soluciones concretas para casos particulares. Las personas particulares que no ganamos dinero con los programas no podemos ni pensar en contratar a nadie. Pero las empresas contratan programación porque lo que necesitan son programas que les permitan hacer su trabajo.

El cambio de modelo es algo que realmente ya se da. Es pasar de vender programas a vender servicios. Si tu programa es de programación libre o de código abierto[14], es mucho más fácil que consigas colaboraciones de gente que quiere mejorar el programa. De gente que sabe programar y que corrige fallos o aporta mejoras. En realidad, sólo se pueden vender servicios a quien puede pagar por ellos. Porque lo mismo pasa con los programas. Sólo te pagará quien pueda. Se puede obligar al pago por el programa. Pero en el peor de los casos, el programa tendrá menos usuarios. En cambio, siempre se paga por servicios. Si tu programa tiene más usuarios, más fácil será que la gente se interese por tus servicios.

Con la programación libre, también se consigue algo muy importante: que la gente aprenda a programar de modo más fácil. Te puede parecer que es algo demasiado especializado. Cuando tengas treinta años, todas las personas que te rodeen sabrán a escribir. Es probable que la mayoría sepa mecanografía. Aunque quizá

para entonces sólo se dicte al ordenador. Con seguridad, todos usaréis ordenadores. Sin embargo, ¿no es extraño que programar siga siendo algo de especialista? Es raro, porque saber programar no es ser informático. Igual que saber escribir no supone dedicarse profesionalmente a la escritura.

Vamos a verlo desde otra perspectiva. Hoy nos parece raro que quien haya ido a la escuela no sepa leer y escribir. Lo hará mejor o peor, como todo, pero sabe. Eso no ha sido siempre así. Porque hasta que se generalizó la educación a todas las clases sociales, la mayoría no iba a la escuela. Y no aprendía a leer y escribir con cierta soltura. Con la programación ocurre algo parecido: la mayoría no sabe leer ni escribir código informático. Eso sólo puede cambiar si se enseña programación en el colegio.

Enseñar a programar es bueno por el mismo ejemplo del comienzo de este capítulo: hacer cosas útiles con los ordenadores. A diferencia de la impresora de Stallman, tenemos ordenadores cada vez en más sitios. Saber programar no es sólo poder escribir un programa partiendo de cero. A veces es importante saber cómo funciona, o poder corregir o mejorar algo. Para eso, no hace falta tanta práctica. Aunque el ejemplo no es igual, no es lo mismo corregir una falta de ortografía o añadir una frase, que escribir un libro.

## C LOS RECURSOS COMUNES

Si podemos mejorar los programas que usamos a diario, tenemos un catálogo de recursos comunes más rico. Porque es algo a partir de lo que todo el mundo puede empezar a construir. Con un ejemplo básico, el acceso a papel y bolígrafos es básico si queremos que la gente escriba. Los programas informáticos son como los bolígrafos. Además, en este caso, si los mejoramos en un caso, lo mejoramos en todos. Si conseguimos que un programa —imagina

el que quieras— sea el doble o el triple de rápido, esa mejora afecta a todo el planeta.

Con dos ejemplos reales de uso diario, aunque nos pasen desapercibidos. Son los servidores de páginas de internet. Los dos más comunes —*Apache* y *NGNIX*— tienen entre un 55% y un poco más más del 80% —dependiendo de las estadísticas— de uso. Ambos son de código abierto. Y con algo más básico, el propio protocolo de las páginas de internet. Su creador, Tim Berners–Lee quería usarlo para la diseminación de la ciencia. Quería hacer de internet una biblioteca, de ahí los hiperenlaces. Su idea original era lanzar el *hypertext transfer protocol* —tecleado como `http`— con *GNU GPL*. Si internet hoy puede ser gran mercado, es porque se diseñó y funcionó como una gran biblioteca.

Los recursos comunes son esenciales que pueda haber todo lo demás. Cosas tan sencillas como agua, aire y luz solar son esenciales para que pueda haber más cosas. Todos construimos a partir de esos tres elementos. Constituyen una base común que necesitan un cuidado de todos. Ése es el sentido del respeto al medioambiente: poder beber agua limpia, respirar aire puro y ver la luz del sol. Esa riqueza es común porque nos permite a todos ganar.

No sólo existen recursos comunes naturales. Son muy importantes los recursos inmateriales. Imagínate, la cultura y la lengua. Eso es básicamente lo que se enseña en el colegio. Imagínate que tuvieses que pagar una licencia por aprender a hablar un idioma. O por aprender matemáticas, historia o la materia que quieras. A lo mejor tú podrías, pero quizá otros muchos no podrían. O dejarías de aprender en cuanto supieses lo básico, porque quizá el resto no te lo podrías permitir.

La economía suele dividirse en tres sectores: primario, secundario y terciario. El sector primario es el que ofrece materias primas. Pueden ser materias primas la madera o los tomates. El sector

secundario ofrece productos manufacturados de materias primas. Productos manufacturados de las materias primas anteriores son muebles y salsa de tomate. El sector terciario es el sector que ofrece servicios. Son servicios el alquiler de habitaciones de hotel amuebladas o las comidas en un restaurante. Una economía es más rica cuantos más servicos ofrece. Puedes vender buenos tomates, pero es mejor si tienes buenos restaurantes. Las materias primas y los productos pueden comprarse más baratos que los servicios. Porque los servicios añaden un valor específico —si quieres, único— a productos y materias primas.

### D LOS TRES SECTORES DE LA INFORMÁTICA

Volvamos a la informática y pensemos que el código es la materia prima de una sociedad digital. Por supuesto, puedes pensar que los programas informáticos son un servicio. Así piensan también quienes los venden. Aunque en realidad, cuando tienes un programa no tienes realmente nada. Tienes la posibilidad de usarlo, siempre que sepas cómo. Nada más. Si pensamos que el código es un punto de partida, si es la matería prima, podemos desarollar una economía y una sociedad mucho más rica. Si el límite no lo ponemos justo al principio, llegaremos más lejos.

Imagínate que por cada servidor de internet hubiese que pagar un dinero por licencia del sistema operativo y de los programas de gestión de la información que aloja el servidor. Además, vamos a suponer que apenas se pudiese configurar el servidor. O que lo tuviese que hacer el fabricante de los programas, previo pago. La consecuencia más clara es que internet tendría un desarrollo más lento. Seguro, las grandes empresas y grandes gobiernos pagarían por los servicios más importantes. Muchas universidades y organizaciones sin ánimo de lucro tendrían muchos problemas para publicar no pocas cosas en internet. Incluso muchas empre-

sas sólo desarrollarían ciertos proyectos cuando tuviesen el éxito asegurado —lo que nunca se sabe de antemano—.

¿Es que todo es gratis? No, por supuesto que no. Es que si la réplica de programas es prácticamente gratuita, quizá el valor esté en ofrecer algo más que el propio programa. Por ejemplo, existe un conjunto de programas parecido a *Microsoft Office*, que se llama *LibreOffice*[15] y es de programación informática libre. No funciona exactamente igual, aunque su funcionalidad es parecida. Por supuesto, se podría vender. Pero es mejor vender servicios, como desarrollo de nuevas funciones o asistencia técnica a empresas. De hecho, aunque tenga un gran grupo de usuarios que lo descarga y lo usa gratuitamente, hay personas y empresas que ofrecen los servicios descritos. Incluso los usuarios pueden aportar informes de fallos o mejoras, traducciones, documentación, e incluso el código con las correcciones o mejoras planteadas.

El producto de lo que el código es la materia prima puede ser la asistencia técnica. Sé que es un servicio, pero lo quiero distinguir de los servicios que querría plantear después. La asistencia técnica es instalar el programa y hacer que funcione. También supone arreglar los problemas que puedan aparecer. O confirmar que hay que hacer las cosas de otro modo. No sólo existe este producto en el ámbito empresarial, sino también entre familia y amistades. Por supuesto, no es algo que se cobre con factura. Es una forma de ayuda gracias al saber de la persona que facilita la informática a quienes se les hace más cuesta arriba. Siempre que no se abuse de la confianza, esa ayuda es valiosa. De hecho, el abuso se puede dar porque la ayuda es valiosa.

Los servicios respecto a los programas podrían ser de dos clases: proporcionar la cosa acabada o enseñar a manejar el programa. Supongamos que tienes una empresa o una asociación de cualquier tipo. Quieres tener un logotipo, una página en internet y un folleto explicativo. Dependiendo del tamaño de la empresa o la

asociación —según el número de personas que tengan—, puedes necesitar que te lo haga alguien o que alguien te enseñe a manejar los programas para hacer lo ya está bien en papel.

La chapuza sería pensar que ya lo harás —o lo hará cualquiera— usando *Microsoft Word*. Si no sabes diseñar, tener un ordenador no te va a enseñar. Los folletos del *Dia* —la marca comercial no se acentúa[16]— están diseñados por personas que saben cómo poner las cosas. Si lo haces sin saber, tu organización dará sólo una mala imagen. Así será más difícil que la gente quiera interesarse. También pasa con el ejemplo que tienes entre manos, este libro. Puede hacerse de manera cutre y sin cuidar la composición tipográfica. Pero con eso lo único que ganaría es dificultar la lectura para quien quisiese leer estas páginas. Hacer chapuzas se acaba volviendo contra quien las hace.

En los dos ejemplos —la empresa o la asociación—, te interesa pagar a una persona. Porque el resultado será infinitamente mejor y mucho más rápido. Si no lo es, estarías contratando a un chapuzas. Sería alguien que te toma por tonto y se quiere aprovechar de ti. Precisamente trataría de engañarte porque el servicio que te ofrece la cosa terminada o te enseña a manejar el programa es valioso. Su valor es que merece la pena pagar por él. Porque no siempre tenemos que hacer todo por nosotros mismos. O no siempre tenemos tiempo para aprender a hacerlo.

Con la informática hay una incomprensión que puede ser un peligro grave. Por usar un ordenador no dejamos de tener que saber cómo se hacen las cosas. El caso más claro es escribir. Por muy bien que te pueda corregir un ordenador la ortografía y la gramática, tienes que saber escribir tú con papel y bolígrafo. Porque de otro modo, no sabrás escribir. Lo peor de todo es que así nunca escribirás bien.

Otros ejemplos son la fotografía o la grabación de sonido. Tener una cámara fotográfica y un programa de retoque no quiere decir saber hacer buenas fotos. Porque para eso hay que saber de fotografía. También podrías editar un disco con un ordenador, pero para eso tienes que saber de ingeniería de sonido. Si no, el programa te sonará a chino y no te servirá para nada. Aprender a usar un ordenador es mucho más útil, cuando sabes hacer las cosas sin él.

Resumiendo, el modelo sobre el que se asienta *Linux* es un modelo en que todos ganamos. Del mismo modo que ganamos con todos los recursos comunes, como el agua limpia o la cultura común. El negocio no está en el código —en los programas—, sino en ofrecer servicios. El código es la base sobre la que empezamos a construir. Así podemos conseguir un programa mejor. Porque para que gane yo, no hace falta que pierda otra persona. De hecho, como en los buenos juegos, gano cuando ganan los demás.

# 4 LA PROGRAMABILIDAD

*A La utilidad de los ordenadores B El asistente digital*
*C Órdenes informáticas D La máquina E La tarea humana*
*F La ayuda de la informática*

## A LA UTILIDAD DE LOS ORDENADORES

Los ordenadores son útiles porque son programables. No tenemos que saberlos programar para usarlos. Sin embargo, son capaces de realizar distintas tareas, porque podemos ordenarles que las hagan. Realmente, los ordenadores saben hacer muy pocas cosas. Esas poquísimas cosas que hacen por sí mismos, son extremadamente simples. Por tanto, tenemos que mostrarles cómo hacer el resto desde esas operaciones básicas. Paso a paso, indicándoles cómo tienen que proceder en cada uno de los momentos de la tarea.

Los ordenadores sirven porque podemos programarlos. Eso supone darle órdenes a diferentes niveles. Un sistema operativo es el conjunto de órdenes básicas. Ese conjunto básico de órdenes es necesario para que el ordenador pueda recibir y ejecutar órdenes más avanzadas. De otro modo, cualquier programa tendría que empezar a ordenar desde cero. Sin sistema operativo, habría que inventar la rueda para cada programa distinto. Sería como tenerle que enseñar a una persona un idioma para pedirle algo sencillo.

La programabilidad es la capacidad de esas máquinas lógicas universales que llamamos ordenadores para recibir y ejecutar órdenes. Ahí está toda su utilidad, que no es en absoluto pequeña. La mayoría de las personas sólo hemos de usar ordenadores. No

será necesario que los programemos, pero tenemos que saber cómo están estructurados. Para saber qué podemos esperar de lo que pueden hacer y qué no. Sólo así sabremos sacarles el máximo partido posible.

## B EL ASISTENTE DIGITAL

Un ordenador es un sirviente, porque sólo sabe recibir órdenes y ejecutarlas. No es una persona, sino una cosa. Sólo necesita entender las órdenes y las llevará a cabo siempre que no le pidamos nada contradictorio. Contradictorio es lo que es imposible lógicamente, lo que no puede ser por principio. Un ordenador no es sólo lo que sirve para poner orden, sino que es también aquella máquina a la que podemos mandar. De hecho, un ordenador sólo sabe hacer una cosa: seguir órdenes. Ejecuta tareas lógicas porque entiende las órdenes para realizarlas.

Un ordenador no es una persona, sino una máquina. Sin embargo, te puede ayudar a entender qué es una máquina lógica que recibe órdenes, imaginarlo como una persona. Piensa en un sirviente que sólo entendiese un poco de inglés. A diferencia de cualquier persona, propiamente nunca aprende. Tampoco es capaz de recordar, si bien puede leer lo que ha hecho otras veces. Así, puede seguir con lo que ha hecho antes.

La característica más destacable de este sirviente es que realiza muy rápido muy pocas cosas básicas. Lo mejor es hacerle combinar esas operaciones y conseguir que haga cosas más complicadas. Lo normal sería pensar que este sirviente aprende y que se hace cargo de la situación. Pasaría con una persona, pero no puede pasar con una máquina. Somos nosotros los que nos tenemos que hacer cargo de la máquina. Eso se traduce en que a la máquina hay que anunciarle cómo tiene que hacer todo. Eso es lo que significa en su origen la palabra programa.

Los ordenadores no piensan, ni pueden pensar. Me puedes decir qué es entonces la inteligencia artificial. Pues muy sencillo, es inteligencia simulada. Se trata de dar al ordenador las reglas para que parezca que piensa, pero en realidad no piensa. Eso es programar, decirle al ordenador cómo tiene que resolver todas las situaciones. Un ordenador no sabe improvisar, decidir o cambiar respecto a lo mandado. Puede haber una orden con la que parezca que está actuando por sí mismo. Pero es sólo una simulación, que hace que nos parezca que hace algo solo.

El ordenador sólo calcula. Esa capacidad de cálculo es muy superior a la de cualquier persona. También es mucho más rápido que cualquiera. Con esos cálculos es capaz de entender órdenes básicas. Con las órdenes adecuadas, es capaz de ejecutar órdenes más complicadas. Pero todo tiene que estar claro antes. Todo tiene que estar pensado antes de que el ordenador actúe. Si no sabe qué hacer con una orden —porque no se lo hayamos dicho antes—, la máquina no hará nada.

Un ordenador entiende en un único sentido. Es capaz de ejecutar órdenes porque se las hemos explicado antes. Le hemos dicho qué tiene exactamente que hacer —y cómo— cuando reciba cada orden. Si desconoce la orden, comunicará que hay un error porque ignora el mandato. Si conoce la orden que usamos de modo distinto al especificado, arrojará también un mensaje de error porque no la usamos como sabe que está especificada. Ahí puedes ver que el ordenador sólo calcula, no es inteligente.

## C ÓRDENES INFORMÁTICAS

Una de las ideas básicas más importantes de la informática la expresó uno de los mayores genios de la informática de este siglo: *[c]omputers are good at following instructions, but not at reading your mind*[17]. Afirmar que el asistente digital no se hace cargo de la situación es otra manera de decir que los ordenadores no te leen

la mente. No saben qué les quieres mandar, sino que sólo ejecutan lo que le ordenas correctamente. Y correctamente es conforme a lo que le has especificado antes al ordenador.

En el fondo, una orden informática es algo más básico de lo que piensas. También puede parecer que una orden informática está en un nivel más profundo y más básico de lo que imaginabas. Una orden informática no existe en tu cabeza. Ni siquiera en la lengua común que usas a diario, no importa cuál sea. Son órdenes hechas para comunicar con ordenadores. Igual que para comunicarte con una persona que no hable tu idioma tendrías que aprenderlo antes, con los ordenadores tienes que aprender antes qué puedes decirle y cómo. En ambos casos —con personas y con máquinas—, todo lo que dices no sirve de nada cuando no te entienden.

La diferencia de las órdenes informáticas con un idioma es que las primeras son muchísimo más pobres. El ordenador es tonto, sólo entiende lo más básico dicho de un modo unívoco. La informática no sirve cuando puede entenderse de varias maneras posibles. Que algo signifique varias cosas —así nos hacen gracia las bromas y los chistes— es algo que sólo podemos gestionar las personas. Para un ordenador, múltiples significados a una misma expresión le impiden hacer nada, lo bloquean.

### D LA MÁQUINA

Un ordenador es tu asistente digital. No es inteligente, pero no por eso no deja de ser útil y capaz. Tiene la particularidad de que su velocidad sólo la podrás aprovechar si sabes hacerle llegar qué tiene que hacer. Eso supone que sepas mandarle para que ejecute tus órdenes. El ordenador nunca aprende, aunque puede retomar lo que hayas grabado antes —programas o documentos—.

Un ordenador es extremadamente útil porque es una máquina. No es sólo una herramienta. Espero que la diferencia sea clara. La

herramienta es algo de lo que nos ayudamos para hacer algo. En realidad, una herramienta es sólo una ayuda —más precisa o más fuerte— para las manos. Las herramientas son extensiones de las manos, por más precisas que sean. En cambio, una máquina es aquel aparato que hace algo funcionando sola. La máquina más clásica es la de vapor, porque con su fuerza realizaba diferentes tareas.

Hoy en día, casi todas las máquinas usan electricidad, al menos. Por supuesto, puede haber también herramientas eléctricas. Lo relevante es que te des cuenta de que las máquinas funcionan solas. No se ponen en marcha por sí mismas, pero una vez que están encendidas realizan su tarea automáticamente. Piensa en una lavadora o en un lavaplatos. Dependiendo de cómo se vean, son máquinas que realizan tareas sencillas. No son simplemente un destornillador o un martillo, que necesitan manos que los usen. A diferencia de las herramientas, las máquinas funcionan de modo automático.

Un ordenador es una máquina, como también lo es una calculadora. La diferencia entre ambas es que la calculadora sólo sirve para hacer operaciones matemáticas. Una calculadora es una máquina con una finalidad determinada, ya que sólo puede hacer operaciones matemáticas. En cambio, un ordenador es una máquina que puede hacer cualquier tipo de operación. Así puede usarse como calculadora, reloj, reproductor y grabador de sonido y de vídeo, máquina de escribir… todo depende de que tenga los programas y los periféricos adecuados para cada tarea.

Un ordenador realiza automáticamente tareas que le mandamos. Las hace mucho más rápido que cualquier persona y sin errores. Un ordenador no se equivoca, aunque puede llegar a fallar por problemas físicos. La grandísima mayoría de los fallos de los programas son de de la persona que los escribió —sea un programa concreto o el sistema operativo—. Por eso, un ordenador es

un autómata o un robot. Una vez que le damos todas las instrucciones, funciona sólo. Eso es un automatismo, algo que funciona sólo una vez que se inicia. Como todo, una operación automatizada puede ser más simple o más complicada.

El ordenador es una máquina que hace cosas de modo automático. Necesita suministro eléctrico e instrucciones precisas. En este punto, la diferencia fundamental con una persona es que no tiene autonomía. No decide por sí mismo, no puede tomar nunca la inciativa. En cierto modo, podrías decir que usa los recursos —programas y periféricos— que tiene a su alcance. Pero todo es según las instrucciones que una persona le ha dado. Como máquina contiene herramientas, que usamos tanto las personas, como que puede usar el ordenador para cumplir sus funciones.

## E LA TAREA HUMANA

Con un ordenador, hacemos cosas. O más bien, usamos el ordenador para que nos haga o nos ayude a hacer tareas. Propiamente, el ordenador no tiene que hacer nada para sí. En el fondo, no lo olvides: es sólo una máquina. La tarea es como la existencia, algo netamente humano. Por eso el ordenador trabaja siempre para ti. Cuando trabajas —o te parece que trabajas— para la máquina, probablemente hay algo que estás haciendo mal. O hay algo que seguro que podrías hacer mucho mejor.

El hecho de que el ordenador trabaje para ti, no es vagancia o comodidad. El ordenador no te sustituye, realmente no hace tu trabajo. Tu tarea al usarlo es pensar lo que viene después. O hacer otra cosa, incluso con el mismo ordenador. Excepto los más básicos[18], los ordenadores permiten que hagas más de una tarea al mismo tiempo. Aunque sus recursos no se multiplican. Por tanto, el poder hacer otra cosa cuando el ordenador procesa, está limitado por el tipo de tarea y del ordenador.

Como espero haberte mostrado, un ordenador funciona porque se puede programar. Eso supone que le podemos dar instrucciones de cómo hacer las cosas. Por eso, las primeras instrucciones son de cómo entender el resto de instrucciones. Un ordenador es una máquina útil porque alguien lo ha programado antes. Antes de que uses un ordenador, una persona ha tenido que diseñarlo antes. No me refiero al ordenador completo, ni a una única persona.

La programación es el modo de hacer que el ordenador sea útil. No es sólo que sea fácil de usar, sino que quien lo use pueda sacarle el máximo partido del asistente digital que es. Por poner un ejemplo: imagínate que necesitas copiar archivos de una carpeta. Con el ratón, o incluso con el teclado puedes seleccionar todos y arrastrarlos a donde sea. Si tuvieses que copiar sólo una parte de los archivos —los documentos PDF de hoy, por ejemplo—, tendrías que buscarlos y seleccionarlos uno a uno. O usarías una orden tecleada para ese fin concreto. Ahí radica la diferencia entre que trabajes para el ordenador o que trabaje para ti.

### F LA AYUDA DE LA INFORMÁTICA

Aprender a usar un ordenador es llegar saber cómo hay que mandarle cosas. Con *Linux* —u otro sistema operativo— no tienes que saber programar. Es útil para escribir programas. Saber informática es aprender cómo puedes hacer tareas con ordenadores. La idea básica es que la informática haga lo más posible por ti. Lo importante es que hay muchas tareas que el ordenador puede hacer solo. Porque le podemos dejar procesando, aunque hagamos otras cosas dentro del mismo ordenador.

Tiene sentido aprender informática para que consigas hacer las cosas más rápido. Y con menos errores en lo que sea automático. Hay cosas que tienes que saber hacer tú. Ningún ordenador puede relevarte de pensar y de las tareas más humanas. Por ejemplo, un

ordenador nunca podrá escribir por ti. Incluso aunque quieras que te corrija la ortografía, es muchísimo mejor que aprendas tú a escribir bien. Porque escribir no es una operación informática. El saber es algo humano y en eso la informática no te puede sustituir.

Los ordenadores pueden ayudarte en la tarea. Imáginalos como herramientas de tu mente. Al igual que un destornillador o unos alicates, pueden ser como la imprenta o la calculadora. Ninguna imprenta ha escrito nunca libros, pero ha ayudado a difundirlos. O la calculadora nunca ha generado riqueza, pero ha ayudado a llevar las cuentas de quienes la generaban. Por eso, debes saber cómo pueden potenciar la labor de tu inteligencia. Aunque sabiendo siempre que la informática es un medio muy útil.

# RECONOCIENDO EL TERRRENO

# 5 EN TIERRA EXTRAÑA

*A Hacerse el sueco B El exilio perpetuo C Manejo informático*
*D ¿Conocimientos de especialista?*

Estamos rodeados de ordenadores y los usamos constantemente. Además de los ordenadores de sobremesa y portátiles, también tenemos tabletas, teléfonos «inteligentes» y lectores de tinta electrónica. En algunos casos, incluso la presunta «inteligencia» llega a los relojes. Es arriesgado afirmar que la mayoría no sabe de informática. Aunque nos pasemos el día tocando pantallitas, la mayoría tiene problemas de comprensión básicos. Toda la formación informática es del uso del dispositivo como un juguete. Lo único que se enseña es a aprender a jugar. Es cuestión de tocar en el color adecuado de la pantalla, consiguiendo así el efecto deseado.

Probablemente un ordenador es mucho más que un juguete. Es más que seleccionar en un menú, o tocar tal o cual botón. Éste es el motivo de que pueda tener interés aprender. Sin embargo, para poder aprender hay que reconocer que no sabemos. De que tenemos que tener cierto interés por lo que nos puedan contar. No para memorizarlo y creérnoslo a pies juntillas. Sencillamente, es necesario que escuchemos para que comencemos a aprender. Nos enseñarán lo que quieran, aprenderemos nosotros mismos. Enseñar tiene sentido para que quien escuche pueda aprender más rápido que quien habla. O al menos, que pueda aprender más seguro, porque ya conoce los errores que otras personas han cometido antes.

No se trata de que todo lo que hemos aprendido hasta ahora sea total y absolutamente falso. Simplemente es demasiado par-

cial. Es como ver el mundo por un canuto. Hay muchas más cosas que las que muestra el canuto. Sobre todo, necesitamos una imagen más completa de la informática. Saber usar ciertos programas está bien, pero es insuficiente. Se trata de saber usar un ordenador. De esa manera, no estaremos limitados por lo que un programa concreto nos pueda ofrecer. Sólo así llegaremos a usar la máquina completa y no programas fragmentarios.

## A HACERSE EL SUECO

Si a cualquiera nos dicen que tenemos que irnos a vivir a Suecia, el primer problema que se nos vendría a la cabeza es el idioma. No es cuestión del frío, que sin duda hará mucho en invierno. La mayor limitación que nos encontraremos será el sueco. Si tenemos ciertos conocimientos de inglés, notaremos menos esa falta. Supongamos también que el inglés nos resulte un idioma desconocido. Mientras no aprendamos el nivel más básico de sueco, nuestras posibilidades de comunicación estarán muy severamente limitadas. Si encontramos a alguien que habla español, desahogaremos nuestra frustación por la incomunicación. Aunque esas conversaciones esporádicas no cambiarán nuestra imperiosa necesidad de aprender sueco.

El desconocimiento del idioma limita nuestras posibilidades de interacción humana. No es sólo una cuestión de comunicación. Lo podemos ver con un ejemplo. Alguien nos viene a hacer una visita y decidimos ir a tomar un café con tarta a una pastelería. Nuestra compañía depende de nuestra invitación porque está de visita. ¿Cómo pediríamos la especialidad de la confitería y un café con leche sin lactosa? ¿O un té con bergamota o una manzanilla? Es bastante probable que tuviésemos dos posibilidades. La primera es la carta con el listado de bebidas. Intentando hacernos con la lista, señalaríamos el texto que suponemos que puede ser lo que queremos. La tarta es fácil que nos acerquemos al mostrador con

las tartas e indicaremos las que quieremos con el dedo. Con un poco de suerte, golpearemos suavemente el cristal con las uñas. Incluso quizá dos veces.

Creo que es algo más que casualidad, pero así se reducen las posibilidades de interacción informática conocidas para la mayoría. No cabe duda de que señalar lo que está a la vista es altamente intuitivo. Sin embargo, tiene una gran limitación, porque sólo puedes ver lo que tienes delante. Seleccionar de un menú es tener que saber de memoria las listas. En ambos casos, las opciones son sólo las que se presentan. Es muy difícil hacerse con posibilidades que no se muestran de modo claro. Porque se nos puede ocurrir algo perfectamente lógico, pero que no lo pueda señalar entre lo que tenga a la vista.

### B EL EXILIO PERPETUO

Volviendo a Suecia, cuanto más tiempo llevemos, nos parecerá extraño que no nos entiendan. Aunque seremos conscientes de que no sabemos el idioma, nos puede llegar a sorprender que no nos entiendan mejor. Sólo conseguimos señalar, pero hay cosas que están claras por su contexto. ¿Cómo puede ser que no se den cuenta de cosas tan claras? En la vida real, lo que pasamos por alto es que el contexto no es común, aunque creamos o contrario. Ese es el motivo de que lo que parece obvio en un país, pueda no serlo tanto en otro.

Antes de terminar con el ejemplo de estancia en el extranjero, imaginemos una última posibilidad. Piensa que te dicen que puedes estar ahí todo el tiempo que quieres. Excepto por el idioma, vives bien allí. Se plantearía la siguiente situación con el paso del tiempo. Independientemente de otras cuestiones, cualquier persona se vería en la situación de aprender sueco o irse. En la vida real, incluso aunque puedas hablar inglés —no es una opción en el ejemplo que menciono—, aprender sueco es con el paso del tiem-

po una necesidad mayor. Porque de otro modo, si vas a vivir allí un tiempo, estarás en una extranjería perpetua. No se levantaría nunca la barrera del idioma. Aunque te diesen la nacionalidad.

Eso sería como estar siempre en el exilio, siempre fuera de lugar. No es que la gente tenga otro trato contigo, sino que el idioma pone un límite anterior a otras cosas. La persona que no habla la lengua común del lugar donde vive tiene una limitación. Si con el tiempo no quiere hablarla, le acaba imponiendo —independientemente de lo que piense— esa limitación a los demás. Así seremos extranjeros perpetuos, estaremos siempre fuera de lugar. Desde el ejemplo que me interesa ilustrar, nuestra capacidad de interacción estará severamente limitada. Podremos llegar a un nivel básico, pero de ese punto no pasaremos nunca.

### C MANEJO INFORMÁTICO

Es claro que el ejemplo tiene limitaciones. Ya no sólo porque es raro que nos vayamos a un país extranjero sin conocer nada de la lengua. Ni siquiera sabiendo una tercera lengua común. También es extraño que no aprendamos la lengua del país que nos recibe. Por pura supervivencia, tenemos todos los incentivos del mundo, aunque luego aprendamos ese idioma mejor o peor. Y también es poco habitual que decidamos quedarnos sin saber ni querer aprender nada del idioma de la tierra en la que estamos.

Sin embargo, la informática es más restrictiva. En Escandinavia es fácil que nos entiendan en inglés —o incluso en alemán o francés—. Un ordenador entiende las órdenes que entiende, no se hace cargo de ningún contexto. Si no se lo decimos de modo correcto, no conseguiremos que haga lo que queramos. Podemos decir que los ordenadores son «tercos». Aunque lo pongo entre comillas, porque son sólo máquinas que no pueden leernos la mente. Ni, como el resto de personas, pueden hacerse cargo del contexto

común. Por mucho que nos empeñemos, el ordenador no aprende en ningún sentido. Sólo podemos aprender las personas.

Ese nivel de comunicación precario es el manejo que tienen muchos de ordenadores. O si se prefiere, de dispositivos informáticos. Por mucha inteligencia que pretendamos transferir a los dispositivos informáticos, en realidad los usamos de un modo pobre. O es fácil que no sepamos usarlos. La misma imagen nos lo muestra: o podemos jugar con ellos, o en cuanto nos cambian lo más mínimo estamos perdidos. En el ejemplo de la cafetería, pasaría como si nos encontrásemos el mostrador vacío. Literalmente, nos quedaríamos sin palabras. Y con mucha hambre.

Con un ordenador podemos jugar, podemos usarlo como una herramienta para hacer tareas concretas. Jugar nos facilita la tarea. O más bien, poder jugar nos hace más fácil la tarea informática. Eso no quiere decir que lo que tengamos que hacer lo hagamos mejor. Simplemente el dispositivo informático no será un problema añadido. Como herramientas, los programas son útiles, pero tienen una limitación. Imaginemos que queremos construir un mueble —ya no digo un coche—. Con un destornillador, un martillo o unos alicates podríamos arreglar detalles, pero para hacer un objeto de cierta complejidad, las herramientas no nos valen. Usando el ordenador como herramienta, usando sólo programas, tenemos la misma limitación.

Estar limitados a lo que podemos señalar tiene ventajas al principio. No cabe duda de que es más fácil. Sin embargo se plantea la pregunta de qué pasa si no vemos lo que queremos o necesitamos. Imaginemos una hoja en blanco. Que no tenga nada escrito o dibujado nos ayuda a poder escribir o dibujar en ella. Incluso es esencial que esté en blanco para que podamos usarla. En definitiva, que el folio esté vacío nos permite hacer las cosas como nosotros queramos o entendamos mejor.

Un ordenador es una máquina universal. Con ella podemos hacer muchas cosas, muchas más que movernos entre pantallas. La única condición es que tenemos que saber usarlo. Igual que en la pastelería, puede haber otras cosas que las que se exponen en el mostrador. Pero para siquiera poder tener acceso a ellas —ya no digo para usarlas—, tenemos que saber como invocarlas. Tenemos que saber cómo jugar lógicamente —usando palabras— con ellas. Sólo así usaremos la máquina a la que llamamos ordenador.

### D ¿CONOCIMIENTOS DE ESPECIALISTA?

Manejar un ordenador supone no sólo ir señalando cosas a golpecitos, sino tratarlo de tú a tú. Como veremos en el próximo capítulo, es nuestro asistente digital. Dicho muy brevemente, supone que está para lo que le mandemos. Todo lo que tenemos que aprender de informática es mandar a la máquina. Qué suponen esas órdenes desde un punto de vista general, es lo que trataré en los dos siguientes capítulos.

Como en casi todo en la vida, mandar a un ordenador se puede hacer desde diferentes niveles. Por supuesto, no es lo mismo programar que tener conocimientos de informática. Las capacidades son distintas, no están al mismo nivel. No me interesa aquí saber programar. Sin embargo, en ambos casos se trata de mandar, de exigir al ordenador que haga lo que le indicamos. Es ser dueños y señores de la actividad de la máquina. No es ver a ver qué pasa, ni tampoco como pedir al ordenador como si del oráculo de Delfos se tratase. No hay que esperar que al ordenador le caiga en gracia acceder a nuestras peticiones. Ni tampoco hay que conjeturar qué está pasando realmente en el ordenador. Tenemos que mandar y que la máquina cumpla. Eso es la informática.

Lo que propongo puede suscitar dudas en más de una lectura. Aunque no sea programar, ¿manejar un ordenador así no es para especialistas? No es así, aunque nos pueda parecerlo. Lo voy

a mostrar con dos ejemplos de tareas mucho más difíciles. Toda la sociedad asume que cualquier persona debe poder desarrollar ciertas tareas básicas. Como todo, cada cual lo hará mejor o peor. El hecho de que haya gente que se gane la vida con esas actividades, no supone que nadie puede dejar de hacerlas. Son cuestiones tan básicas como leer y escribir. Una parte muy importante de la educación es que aprendamos a leer y a escribir. Por suerte, en nuestra sociedad la tasa de analfabetización es muy baja.

Para analizar con más detalle, voy a dejar de lado la escritura. Sólo hablar nuestra lengua materna —sea la que sea— es infinitamente más complicado que programar. Ya no digo aprender a usar un ordenador —tarea más fácil—, sino programar. No nos damos cuenta, porque ya hemos aprendido y lo habla todo el mundo. Pongo ejemplos, como saber los géneros —gramaticales— de las palabras, en qué sílaba llevan el acento —de entonación y gráfico—, o cómo se forman sus plurales. De los verbos, saber cómo se conjugan los verbos, cuándo se usan los diferentes tiempos, o aprender a usar los modos. Con un ejemplo estrella para enseñar a alguien que aprenda español: ¿cómo se usa el subjuntivo? Es realmente difícil, porque lo comprobamos con personas que no lo aprendieron de niños. Quienes lo sabemos, lo usamos sin más problema.

En realidad, la primera dificultad de usar ordenadores, de mandarles, es hacernos cargo de que son máquinas simples. Sólo procesan órdenes de una manera que para una persona es demasiado sencilla. Son mucho más rápidos, por pura capacidad de cálculo. Aunque lo simulen, no entienden nada, porque no pueden hacerse cargo de nada. Son capaces de ejecutar lo que les pidamos según un código muy sencillo. Todo lo demás lo podremos hacer las personas, pero no las máquinas.

# 6 EL ASISTENTE DIGITAL

*A Qué y quién B La característica básica C La máquina lógica universal*
*D Inteligencia artificial*

Un ordenador es un asistente digital. Es un aparato que recibe nuestras órdenes y las procesa ejecutándolas. Sólo puede procesar aquéllas que le hemos formulado correctamente. No es que propiamente las entienda, procesa de un modo muy determinado la información que recibe. Si no le proporcionamos los datos —las órdenes, en este caso— de modo que pueda procesarlos, da un mensaje de error y se para. El error no es una equivocación de la máquina, sino de la persona que ordena. La grandísima mayoría de los fallos de órdenes informáticas —a todos los niveles— son órdenes o datos suministrados de una forma que la máquina no puede procesar. Dicho mal y pronto, no sabe qué hacer con ellos.

## A QUÉ Y QUIÉN

Un ordenador es una máquina que trabaja a una altísima velocidad. Está a nuestra disposición, dado que su única finalidad es ejecutar aquello que le ordenamos. Ésa es la razón que permite afirmar que es nuestro asistente digital. Es rápido, pero muy poco capaz. No son dos características contradictorias, como mostraré después. Quizá lo más relevante es algo que es incontestable: un ordenador es algo, no alguien. Si bien metafóricamente podemos hablar de la máquina atribuyéndole características de una persona, se trata de una cosa. De hecho, ni siquiera es algo vivo. Es importante no perder de vista esta premisa básica en un aspecto decisivo. Los ordenadores ni pueden crecer, ni mucho me-

nos pueden aprender en sentido estricto. Nuestra comunicación informática está determinada por esta condición fundamental.

La rapidez de un ordenador es difícil de comparar con la de una persona. Opera muchísimo más rápido que cualquier persona, si bien las operaciones que puede realizar son también infinitamente más simples. Si realiza operaciones complejas es porque le ordenamos cómo tiene que hacerlas desde los pasos que las componen. En realidad, un ordenador no es capaz de hacer nada que una persona no haya diseñado antes. Ese diseño es la programación informática, que nos permite usar la máquina. Es la programación la que «instruye» al asistente digital. No es que lo eduque, sino que determina el proceso de órdenes más complejas desde las totalmente simples.

Nuestro asistente digital entiende una versión muy básica del inglés. En realidad, tampoco es tan distinto del español, porque no se trata de comunicación completa en inglés escrito. Inglés y español proceden del latín, de ahí que la diferencia sea pequeña en la parte común a los dos idiomas[19]. Un nivel básico de inglés será suficiente para podernos comunicar con la máquina. Con lo que hemos aprendido de inglés en la enseñanza obligatoria[20] es suficiente. La parte de aprendizaje no descansa tanto sobre las órdenes, como en entender qué es lo que tenemos que conseguir que haga el ordenador. El aprendizaje es real, porque no se trata simplemente de comunicar nuestros deseos. El ordenador es nuestro asistente digital, no nuestro siervo o esclavo —no es inteligente de ningún modo—.

Aprender a manejar un ordenador es aprovechar la altísima velocidad de la máquina, sabiendo dejar de lado las limitaciones que tiene. En cierto sentido, programamos el ordenador. No me refiero a escribir programas, le ordenamos lo que tiene que hacer ahora. Aunque también le ordenamos lo que tiene que hacer en un futuro, del mismo modo que programamos un horno. Le pres-

cribimos qué tiene que empezar a hacer cuando terminemos de enunciar la orden. A diferencia del horno, en que ponemos un tiempo de comienzo y de final de actividad, al ordenador lo programamos haciéndole trabajar de manera autónoma. Por ejemplo, que descargue un archivo, que compruebe su integridad y que se apague el ordenador si todo está bien —o que emita un pitido, si la descarga o la integridad fallan[21]—.

La informática es un modo de comunicación bidireccional entre la persona y el ordenador. No sólo son las órdenes a la máquina, también están los resultados que ésta arroja. Pueden ser de varios tipos. El primer grupo de mensajes es el resultado propio de la orden, como los datos de la fecha si se la preguntamos al ordenador. No menos importantes son los mensajes del segundo grupo, que son los mensajes de error. En caso de error, el ordenador anuncia que no puede procesar correctamente lo que la persona le pide. Como en todo lo demás, según cada mensaje de error, puede verse cuál es su causa e investigar cómo puede corregirse. El tercer grupo de mensajes serían aquellos en que el propio sistema pide más datos a la persona para poder dar un resultado.

Con la descripción anterior, no pretendo una clasificación completa de los mensajes de la máquina a la persona. Sólo ilustro qué nos puede «decir» la máquina. El resultado final de una orden puede ser los datos ofrecidos en pantalla, el envío de información o la generación de un archivo —entre otros—. Los mensajes de error pueden ser de muy diferentes clases, desde el propio programa hasta el mismo sistema operativo. Como en el grupo anterior, también pueden ser más simples o más complejos —independientemente de que el error sea del programa o del sistema operativo—. La petición de datos necesarios para el proceso de lo ordenado puede también diferentes niveles de complejidad.

En la práctica, una de las órdenes más básicas sería preguntarle la hora al ordenador. En *Linux* sería:

```
date
```

El resultado que me da *Fedora* en el momento que lo tecleo en la ventana de órdenes es:

```
Wed Dec 14 19:14:22 CET 2016
```

Como ya he explicado, configuro *Linux* en inglés para informar mejor de posibles fallos o sugerencias de mejora. Si instalas *Fedora* en español o en otro idioma, el mensaje con el día y la hora te saldrá en ese idioma[22].

Un modelo de mensaje de error sería el siguiente. Usando la misma orden `date`, quizá queramos saber la versión de ese programa. El modo correcto sería:

```
date --version
```

Esa orden nos mostraría datos de la versión[23]. Aunque el mensaje será muy diferente si escribimos lo siguiente:

```
date --versión
```

El mensaje de error es el siguiente:

```
date: unrecognized option '--versión'
Try 'date --help' for more information.
```

Antes de nada, lo único que tendrías que aprender de memoria es que para un ordenador no existen letras. Aunque parezca increíble —no olvides que no aprenden a hablar, sólo usan códigos—, para los ordenadores no existen letras. *O*, *o*, *Ó* y *ó* son la misma letra en español, aunque tenga tilde. Para un ordenador se trata de cuatro caracteres distintos. Por eso, `version` y `versión` no son lo mismo. Serán la misma palabra para una persona —aunque la primera esté mal escrita en español[24]—, pero son cadenas de caracteres distintas —así procesa la máquina— para un ordenador.

El mensaje de error del programa es que no sabe qué opción es `--versión`. El propio programa —por diseño de su programador— te aconseja que busques con la opción `--help` cuáles son las opciones disponibles.

Las peticiones de datos para proceso posterior se refiere a órdenes un poco más complejas que la que acabo de usar. Un ejemplo muy común es la identificación con nombre y contraseña en cualquier servicio a través de internet. Según quién seas, el programa te da acceso a diferentes datos y te permite diferentes operaciones.

### B LA CARACTERÍSTICA BÁSICA

Si el ordenador es una máquina con gran capacidad de cálculo, pero no es inteligente, cabe preguntarse qué es. El asistente digital en el fondo es un automatismo, un grandísimo sistema lógico que funciona de modo automático. No deja de ser una máquina, porque carece de autonomía en sentido estricto. No toma decisiones, aunque podemos programar una máquina para que las calcule. Aunque decidir no es calcular. Sólo en un sentido muy metafórico, podemos decir que «el ordenador decide». Porque realmente decidir y elegir son acciones humanas. ¿Podemos delegarlas en ordenadores? Estrictamente, podemos encomendarles su cálculo según parámetros establecidos con anterioridad. Sin embargo, la decisión o elección ya la hemos tomado antes al someter el proceso a un programa informático.

Atribuir cualidades humanas a los ordenadores conlleva un riesgo. El peligro consiste en no distinguir qué hacen las máquinas y qué hacemos las personas con ellas. El ejemplo anterior de la autonomía es claro. Quizá haya muchas situaciones en las que sea deseable no elegir, sino exclusivamente calcular una valoración numérica —o quizá resulte más cómodo—. Aunque en ese ejemplo, porque las personas renunciemos a nuestra autonomía, no se la entregamos a los ordenadores. La característica definitoria de un

ordenador es el automatismo, que es lo contrario a la autonomía. Un ordenador puede trabajar solo. ¿Eso no sería autonomía? No lo es, porque el ordenador trabaja procesando órdenes y datos que recibe. Si carece de ambos, no realiza proceso alguno —hablando mal y pronto, no hace nada—.

Un ordenador sólo puede puede ejecutar las órdenes recibidas. Por supuesto, cargar un sistema operativo —como *Linux*— y mantenerlo para que responda supone ejecutar órdenes. El automatismo del ordenador es la ejecución de órdenes lógicas de muy diferentes niveles. La ejecución que realiza es el proceso de órdenes de muy diferentes niveles. Porque no es igual una orden de configuración de las diferentes partes del sistema, que interactuar con la persona en las distintas peticiones que pueda hacerle. Las órdenes correctas las lleva a cabo sin necesidad de intervención externa.

## C LA MÁQUINA LÓGICA UNIVERSAL

El asistente digital al que me refiero en este capítulo, puede describirse de cuatro maneras: como autómata, máqina, robot o androide. Cada una de esas maneras —como trataré de mostrar— indica aspectos diferentes y complementarios entre sí. Esas cuatro perspectivas dan como resultado una descripción más detallada de un ordenador. Ese mejor conocimiento del ordenador es relevante para una mejor comunicación con —o un mejor manejo de— la máquina.

### I AUTÓMATA

«Autómata» proviene del griego αὐτόματον, adjetivo que describe lo que actúa por sí mismo. No se trata de una persona, que actuaría por propia voluntad. Ni propiamente se refiere a un ser vivo, que actuaría de modo natural. Obviamente, cuando se refiere a cosas, se trata de un ingenio que parece que actúa por sí mismo.

Aplicado a la informática, un autómata es aquél dispositivo que es capaz de actuar. Esa capacidad no llega a ser acción propia, sino que realiza las órdenes recibidas. La idea básica del autómata informático es que es el perfecto ejecutor de las órdenes recibidas. Su perfección radica en la velocidad muy superior a la de cualquier persona respecto a las acciones propias. Debido también a que propiamente sólo calcula, no tiene posibilidad de equivocación. Puede haber llegar a haber errores, pero son fallos del sistema —errores en el funcionamiento—. Sin embargo, no son equivocaciones en la ejecución, que son propias de personas[25].

El autómata actúa solo una vez que ha sido programado. Ésa es la gran ventaja del uso de ordenadores, que no necesitan supervisión para la ejecución de la tarea. Aunque las órdenes puedan mejorarse, o queramos que cambien el modo en que realizan su tarea. Las órdenes no son perpetuas, podemos cambiarlas para futuras ejecuciones.

## II MÁQUINA

«Máquina» viene del latín *machina*, que a su vez viene del griego μηχανή. Eso equivaldría a afirmar que «máquina» proviene de «mecanismo». La máquina es aquel artificio que se emplea para aprovechar una fuerza. Se distingue de la mera herramienta porque es algo más que un mero instrumento. En cierto sentido, la máquina es la que inicia su funcionamiento automático con el botón de encendido, mientras que la herramienta es un instrumento que necesita habitualmente de la mano. De este modo, la máquina formaría un todo articulado e independiente, que constituiría su mecanismo. Y la herramienta —incluso eléctrica— vendría a ser una extensión de la mano.

Independientemente de la electrónica, diremos que una lavadora es una máquina y no una mera herramienta. A día de hoy, una lavadora —no la máquina, sino su lavado— sólo hay que

prepararla y recogerla, lava sola tras haber pulsado el botón de encendido. Una batidora o un taladro no son máquinas, sino herramientas. Aunque puedan contener incluso electrónica, no dejan de ser instrumentos para una mano humana[26]. Si las dejásemos funcionar solas, es muy fácil que resultasen peligrosas.

¿Qué fuerza aprovecha un ordenador? Articula capacidad de cálculo. Si se quiere en términos más mécanicos, sería la electricidad y la lógica. Que pueda ofrecer interacción no quiere decir que sea una herramienta. Es sencillamente que puede asumir órdenes de modo sucesivo, según resultados de operaciones anteriores. Es más compleja que el encendido de la lavadora o un horno microondas. Por eso puede realizar tareas mucho más complejas.

## III ROBOT

A día de hoy, «robot» y «androide» son palabras prácticamente sinónimas en nuestro idioma. Si las consideramos desde sus orígenes, apuntan a aspectos distintos. La palabra «robot» viene de la lengua checa, del escritor de ciencia–ficción Karel Čapek[27]. La palabra checa es *robota* que significa el trabajo debido al señor feudal. Un robot es aquella máquina que cumple órdenes.

Visto desde la persona, un ordenador es aquel aparato del que tenemos que programar su acción. Hemos de prescribirle qué ha de realizar y cómo. Por supuesto, no tenemos que inventar todo desde cero, porque el ordenador ya está funcionando. Si bien las más básicas no dejan de ser órdenes que el ordenador ha de cumplir. Su trabajo nos es debido a las personas que los usamos. Si somos sus dueños y señores —no conviene olvidar que se trata de una cosa—, tenemos el mayor interés en comunicarnos de la manera más eficiente posible. Es el único modo de conseguir el mayor rendimiento en su ejecución de nuestras órdenes.

## IV ANDROIDE

«Androide» tiene su origen en el adjetivo griego ἀνδροειδής, que significa de apariencia humana[28]. No es coincidencia de que el sustantitvo en inglés lo haya escogido *Google* como denominación comercial de su sistema operativo. Como nota informativa, está desarrollado a partir de *Linux*. Androide es lo que parece humano, pero realmente no lo es. De este modo, sólo podremos comunicar de una manera muy limitada con el ordenador. Pero si nos limitamos a lo que la máquina «entiende»[29], es altamente eficiente.

En sentido propio, un androide tiene una estructura física parecida a una persona. Su apariencia simula la del cuerpo humano. Sin embargo, cabe entenderlo de otro modo, como ejecutor de tareas. No se trata tanto de que sea capaz de realizar tareas físicas, sino de que cumpla lo que le mandamos con órdenes escritas. La apariencia humana del androide sería la simulación de la inteligencia. No desde luego en el sentido abordado en *6.D. Inteligencia artificial*, sino como interlocutor capaz de comunicación. En un sentido matemático de comunicación[30], se trata del envío de un mensaje de emisor a receptor usando un código común.

El ordenador es capaz de recibir órdenes y llevarlas a cabo. Es extraordinariamente eficiente, no sólo por su rapidez, sino porque no se equivoca nunca. Está limitado en la comprensión, porque sólo es capaz de gestionar una versión muy reducida del idioma inglés. Sin embargo, esas características lo convierten en el asistente digital perfecto. Está a nuestra disposición siempre que tengamos electricidad con la que alimentarle. Sólo tenemos que saber asignarle la tarea.

## D INTELIGENCIA ARTIFICIAL

Ya he anunciado los peligros de asimilar los ordenadores a aspectos de las personas, o de atribuirles características humanas.

Uno de los ámbitos más delicados es la inteligencia. Aunque la cuestión es mucho más rica de lo que puedo desarrollar aquí, hay un punto esencial que es importante mencionar. No existe algo así como un sentido común a la inteligencia humana y la inteligencia artificial. Dicho de modo muy parecido, los ordenadores no son inteligentes del mismo modo en que lo son las personas.

En el fondo, los ordenadores son inteligentes en sentido metafórico y las personas son inteligentes en sentido propio. Desde luego, podemos llegar a programar ordenadores para que lleguen a conversar como personas. Podremos llegar a fabricar androides[31] con apariencia totalmente humana. Por supuesto, carece de sentido dudar de que podremos simular la conducta y la apariencia humana. Sin embargo, eso no las convierte en reales. Las máquinas no son personas, ni tienen características humanas, aunque las puedan simular de modo perfectísimo.

No es sólo una cuestión de sentimientos. O incluso de autonomía de la voluntad, de que exista un querer real que no sea mero deseo —imposibles ambos para un autómata—. La diferencia está entre saber realmente y poder operar al nivel que sea «con amplísimos conocimientos» sin entender absolutamente nada. De otro modo, tendríamos que reconocer que las estanterías de las bibliotecas son sabias.

En el fondo, toda la crítica a una idea más o menos fuerte de inteligencia artificial es sencillamente que no podemos pretender que las calculadoras sepan aritmética. De un modo más complejo, los ordenadores son calculadoras binarias. No entienden nada de lo que procesan. El saber humano exige comprensión de los conocimientos. De otro modo sólo existe ignorancia, aunque pueda estar revestida de apariencia de saber.

# 7 LA COMUNICACIÓN REAL

*A Una petición común B ¿Qué pasa aquí? C Las cosas se complican*
*D Posibilidades más complejas E No hay tal disyuntiva*
*F ¿Por qué un código propio?*

Este capítulo pretende mostrar cómo funciona el ordenador en su nivel más básico. En cierto modo, las personas jugamos con la metáfora del escritorio. Tocando en las partes coloreadas de la pantalla —con cualquier tipo de puntero: ratón, lápiz o nuestro propio dedo— simplificamos un modo de comunicar mandatos al ordenador. Como en los juegos de niños, pinchando una o dos veces en las distintas imágenes, todo sucede como por arte de magia[32]. En realidad, el sistema traduce lo que hacemos con gestos a órdenes que pueda cumplir.

De la perspectiva explicada en el párrafo anterior resulta que las personas operamos informáticamente de un modo distinto al que pensamos. No con los gestos en pantalla, sino con las órdenes reales. Es como si en Suecia tradujesen nuestras señales a frases en sueco. Manejamos los ordenadores con los mandatos que el sistema envía al ordenador, no con gestos en la pantalla. De ahí el interés que, al igual que es muy útil saber sueco si vives en Suecia, nos dirijamos al sistema usando sus órdenes propias.

## A UNA PETICIÓN COMÚN

Cuando enciendes un ordenador con *Linux* instalado, acabas en el escritorio del usuario que se haya introducido en el sistema. Esto no es propio de *Fedora*, ni siquiera de *Linux*, cualquier sistema operativo con un sistema de ventanas hace lo mismo. Suponga-

mos que hay un documento de un procesador de textos o una hoja de cálculo. Los nombres pueden ser `documento.odt` y `contabilidad_2016.calc`. ¿Cómo sabes qué son? Por el icono que va encima del nombre.

El icono muestra también la información que el sistema tiene sobre el tipo de archivo. No es sobre un documento sin más, sino sobre su tipo de archivo. Así sabe cómo tiene que manejarlo. Por ejemplo, con qué programa abrir ese documento o cómo imprimirlo —si es que ese tipo de archivo admite impresión[33]—. Eso se muestra en la imagen del icono. Por esa imagen sabemos de qué tipo de archivo estamos hablando: de un documento de *LibreOffice Writer*, de un documento PDF, de un archivo de sonido en formato MP3, o de un archivo de vídeo en formato MP4.

Por eso tenemos que preguntarnos, ante la siguiente lista de archivos que tuviésemos en nuestro escritorio:

- `documento.odt`
- `libro-aventuras.pdf`
- `canción.mp3`
- `vacaciones.mp4`

¿Qué pasa cuando pinchamos dos veces sobre ellos? Para responder a esa pregunta, tomo el primer archivo de la lista. *Fedora* haría lo siguiente:

```
libreoffice documento.odt
```

O quizá, si usas una versión de *Fedora* que no tiene *LibreOffice* instalado[34], el mandato al sistema sería:

```
abiword documento.odt
```

## B ¿QUÉ PASA AQUÍ?

El ordenador no hace magia, de eso podemos tener la seguridad más absoluta. No puede hacer nada que nadie haya preparado antes para que llegue a ocurrir. Sin mucha más experiencia, viendo lo que haría con los tres últimos documentos al pinchar sobre ellos —cada línea es uno de ellos—:

```
mupdf libro-aventuras.pdf

mpv canción.mp3

mpv vacaciones.mp4
```

La estructura común es: `programa archivo`. Se trata de abrir cada tipo de archivo con su programa propio. ¿Cómo sabe *Fedora* de qué tipo es cada archivo y qué programa ha de asignarle? Cualquier sistema operativo lo hace por la extensión. Es lo que viene después del último punto en el nombre del archivo. En los ejemplos serían `.odt`, `.pdf`, `.mp3` y `.mp4`. Habitualmente suelen ser tres letras, pero hay extensiones de cuatro caracteres —como `.opus`, un formato de compresión de sonido— o incluso más —como `.gnumeric`, un programa para manejo de hojas de cálculo[35]—.

## C LAS COSAS SE COMPLICAN

La situación que acabo de describir es la perfecta, por dos motivos. El programa está instalado en el sistema operativo. Y la vinculación de la extensión del tipo de archivo con el programa está hecha. En la mayoría de los casos puede que sea así, pero puede que en alguno no sea así. Esta particularidad no es propia de *Linux*. Pasa en cualquier sistema operativo con entorno gráfico. Tampoco es extraño que la instalación de un nuevo programa reasigne extensiones a ese nuevo programa. Como todo, puede ser que la nueva asignación sea mejor, pero no es imposible que sea nefasta[36].

Pinchar sobre archivos o iconos sin asignación previa nos daría más problemas que soluciones. Porque el sistema no entendería qué estamos haciendo. Es como si en la pastelería sueca golpeamos sobre el cristal de una bandeja vacía en la que hace tres meses había una tarta de chocolate que nos había gustado mucho. En Suecia puede que lleguen a averiguar a qué nos referimos. Seguro que un ordenador que nunca lo hará. Con suerte puede pedir que se instale otro programa de una lista según la extensión. Pero si la asignación de una extensión es errónea, no tenemos más remedio que corregirla.

Por supuesto, que pueda haber programas sin instalar o programas mal asignados a las extensiones no constituyen motivo para aprender a usar la línea de órdenes. Sería una pésima excusa. Porque los programas hay que instalarlos igual y las malas asignaciones hay que corregirlas. Lo que quiero mostrar es que lo que pretendes hacer con el puntero, en realidad lo estás haciendo con una orden escrita.

### D POSIBILIDADES MÁS COMPLEJAS

Pinchar en la pantalla tiene limitaciones también en un sistema en que todo esté correctamente configurado. Por ejemplo, imaginemos que quieres convertir todos los archivos de un directorio —una carpeta en el entorno gráfico— a formato PDF. Supongamos que todos son documentos de *LibreOffice Writer*. La manera normal de hacerlo en un sistema de ventanas sería abrir el documento `.odt`, pulsar el botón de convertir a PDF y cerrar la ventana. A primera vista, con un documento estaría bien, con diez sería un poco pesado, con veinte empieza a ser engorroso y a partir de cien documentos esa conversión es un problema.

Con un mínimo análisis, con más de cuatro archivos estarías perdiendo el tiempo. Ahora mismo te digo por qué. Como tuvieses que convertir mil archivos del mismo modo, tendrías que descu-

brir el modo de que lo hiciese todo el ordenador. En todo caso, pierdes el tiempo porque estás haciendo una tarea que puede hacer el sistema y es imposible que tú la hagas más rápido que la máquina. Además, así podrías estar haciendo otras cosas, incluso en el mismo ordenador.

Con la mayor lentitud y la dedicación a esa tarea, una persona sólo podría introducir errores. La manera de dejar que la máquina lo haga por nosotros sería:

```
unoconv -f pdf 1000-archivos/*.odt
```

Hacerlo a mano no será razonable desde ningún punto de vista. Sólo el desconocimiento nos llevaría a no dejar que lo haga el ordenador. Puede parecer un ejemplo extremo, pero pondré uno que me he encontrado hace no mucho de pura casualidad. Se trata de un libro de casi mil páginas, con muchos títulos de partes, capítulos, secciones y subsecciones. El índice general estaba hecho a mano. Si por cualquier motivo hubiese que añadir una página después de la página diez, todos los números de páginas habría que haberlos escrito de nuevo. Suponiendo que todos los números de página están correctamente copiados.

No todo son tareas encadenadas, incluso de otros tipos. Sencillamente unir varios archivos de sonido en uno, quitar los primeros tres minutos de un vídeo o convertir un documento de *Microsoft Word* a documento ePub es mucho más fácil con una orden escrita. Dependiendo del programa, no sólo es más sencillo, sino que puede ser la única manera posible. El modo gráfico es sólo una simulación, muy útil en muchos casos. Pero no es ni el único, ni el modo más básico de hacer las cosas informáticamente.

### E NO HAY TAL DISYUNTIVA

*I want it all and I want it now* —«lo quiero todo y lo quiero ahora»—, aunciaba Freddie Mercury hace ya un cuarto de siglo[37].

En informática podemos tenerlo todo. Todo aquello que no sea imposible, lo que no se sea lógicamente contradictorio. En este caso, no se trata de línea de órdenes contra un entorno gráfico de escritorio. Hay cosas que haremos mucho mejor en entorno gráfico y otras que con líneas de órdenes. Lo importante es que aprendamos. No es una cosa o la otra, es usar la mejor manera en cada caso.

Habrá mucha gente que se pregunte por qué es necesario aprender. En realidad, pueden argumentar, debemos de hacer las cosas de la manera que sea mejor para cada cual. Es cierto que a mucha gente le es más cómodo usar los ordenadores con pinchazos —de ratón—. Siento decirlo, pero mucho me temo que se debe a pura ignorancia. Usan así los ordenadores no porque quieran, sino porque no pueden hacerlo de ningún otro modo. Es importante ampliar las posibilidades. Ése es el sentido de aprender informática. Cuando tengamos más capacidad, podremos pensar en elegir mejor. Sólo sabiendo estaremos en condiciones reales de decidir. Con la ignorancia nos vemos obligados a lo que tenemos delante.

Tenemos que entender y saber para ver qué es lo mejor en cada caso. Es complicado ver internet en un navegador que no sea visual, por ejemplo. Igual que es menos cómodo tener que descargar o subir una lista de más de diez archivos en modo visual. En el primer caso, las páginas de internet son visuales, incluso aunque tengan fundamentalmente texto. En el segundo caso, necesitamos una orden general que lo haga de una vez y no de uno en uno[38]. Conforme aprendamos más, según ganemos más experiencia, es fácil que prefiramos modos más directos. Pero tenemos que aprender para poder avanzar.

En el fondo, no se trata tanto del modo concreto del uso del ordenador. Aunque suene raro, se trata de comunicar con el ordenador del modo más eficiente. Porque es un asistente que cumple órdenes. Según sea nuestra capacidad de comunicación podremos

realizar más o menos tipos de tareas. Si sólo podemos indicar y no podemos enseñar al asistente —porque no puede aprender—, es lógico que las tareas tengan que ser muy básicas. Sin embargo, estamos perdiendo capacidad para resolver tareas del ordenador. Mientras no consigamos comunicarnos mejor con él, desaprovecharemos las posibilidades más propias de la informática.

## F ¿POR QUÉ UN CÓDIGO PROPIO?

Quizá podamos pensar que tratar de comunicar con el ordenador con su código propio sea excesivo. Una manera de hacerlo efectivamente es decir que es demasiado difícil. Otra manera es engañarnos argumentando que no es necesario. En todo aprendizaje, además de nuestra capacidad, también importa el modo en que nos hagamos con lo que no conocemos. Con buenas explicaciones, ese camino lo recorreremos antes. Con explicaciones deficientes, es fácil que nos perdamos. En ambos casos, el camino tenemos que recorrer. Si nos perdemos, nos han guiado mal, pero no abandonamos la necesidad de aprender.

Conviene que nos pongamos en otra situación de comunicación cotidiana con la informática. Por no volver a la restauración, vayamos a un supermercado. Si señalamos, conseguiremos muchas cosas, no cabe duda. Ahora bien, ¿qué pasa si queremos panecillos de centeno y no los vemos? A lo mejor no los tienen, aunque también podría ser que les lleguen dentro de dos días o semanas. ¿Cómo podemos hacernos cargo de esa situación si no hablamos y sólo señalamos? Si no inhumana, una existencia adulta sin comunicación sería innecesariamente dura.

Con la informática, la comunicación es una cuestión de pura y simple eficiencia. Si tenemos que trabajar con ordenadores —ése parece ser el destino del siglo XXI—, es necesario que aprendamos a usarlos del modo que sea más ventajoso para nosotros, para las personas. No es cuestión de comodidad inmediata, porque quizá

lo más cómodo que fuese directamente no hacer nada. Se trata de sacar el máximo partido con el menor tiempo invertido. Si es una máquina a nuestra disposición, hay de maximizar el rendimiento del esfuerzo humano. Por eso, para saber manejar una máquina, es importante saber antes cómo funciona realmente.

# NOTAS

1 Por supuesto, la cita es irónica. Pero no hay ninguna errata: 10 debe leerse «uno cero» y no «diez», porque es sistema binario, no decimal. En sistema decimal es el número dos. Los ordenadores son binarios.

2 Puede ser en otro ordenador. Aunque en ese caso, puede llegar a ser que no funcione y que no lo sepamos.

3 La operación simple abrir o cerrar los nanotransistores que componen el procesador. Así funciona a nivel físico y eso se traduce en diferentes operaciones informáticas básicas del procesador.

4 Hace casi veinte años, había unas agendas electrónicas que funcionaban así. Perdías toda la información si te quedabas sin batería y no habías hecho copia de seguridad de los datos.

5 La imagen —como todas— es limitada. No caben dos manos, sino que es sólo una. Además, porque con la otra mano tendrías que sujetar el cesto.

6 De hecho, cuando un ordenador va lento porque accede mucho al disco duro, es fácil que tenga poca memoria de trabajo para el procesador que tiene.

7 Sobre este punto, puedes leer el libro S. Williams, *Free as in Freedom*, disponible en `https://en.wikisource.org/wiki/Free_as_in_Freedom`. En `https://tools.wmflabs.org/wsexport/tool/book.php` puedes exportarlo a varios formatos de libro digital. La diferencia entre la primera y la segunda versión es que en la segunda hay correcciones y anotaciones del protagonista del libro, Richard Stallman.

8 Incluso en la Carta de los Derechos Fundamentales de la Unión Europea, se incluye como derecho a la propiedad, en el artículo 17.2 (`http://eur-lex.europa.eu/LexUriServ/LexUriServ.do?uri=OJ:C:2007:303:0001:0016:ES:PDF#page=6`):

> Se protege la propiedad intelectual.

El texto original en inglés es el siguiente (`http://eur-lex.europa.eu/LexUriServ/LexUriServ.do?uri=OJ:C:2007:303:0001:0016:EN:PDF#page=6`):

> Intellectual property shall be protected.

9 La excepción son las marcas comerciales, que se protegen por tiempo indefinido. Pero el motivo de esa protección es evitar la competencia desleal y el fraude a los consumidores. O lo que es lo mismo: que cuando compres aceite *Aceitito Rico* —por poner un ejemplo que no existe, hasta donde sé—, te lleves ese aceite y no otro cualquiera.

10 Desde luego, no lo son Estados Unidos. En la Unión Europea no lo eran hasta la aprobación de la Carta de los Derechos Fundamentales de la Unión Europea.

En realidad, desde la Constitución estadounidense hasta la Carta de los Derechos Fundamentales de la Unión Europea hay un cambio radical. No te preocupes, éste no es el lugar para explicarlo.

11 Puedes encontrar una buena explicación de la licencia y de las diferencias entre las versiones en `https://en.wikipedia.org/wiki/GNU_General_Public_License`.

12 Los programas que uso para generar el documento ePub —`pandoc`— y el documento PDF —ConTEXt— están también bajo la misma licencia. Son sólo dos ejemplos entre miles.

13 Como explica el proyecto *GNU*, en la página `https://gnu.org/philosophy/free-sw.html`.

14 En inglés, *open source software*. Son expresiones sinónimas, aunque no exactamente iguales.

15 Está disponbile para descarga gratuita en `https://www.libreoffice.org`.

16 Para que no te quepan dudas: `https://www.dia.es`.

17 D.E. Knuth, *The TEXbook*, Addison–Wesley, p. 9.

18 Son ordenadores también —como ya te he dicho— tabletas, teléfonos «inteligentes» y demás dispositivos móviles, de los que están excluidos los ordenadores portátiles. En los sistemas operativos de esos dispositi-

vos, no se permite que el usuario pueda manejar simultáneamente varios programas.

19 En la parte no latina del origen del inglés, la diferencia será mayor.

20 En principio, me refiero a España. Aunque supongo que por la proximidad a Estados Unidos, el nivel de inglés en Latinoamérica será mayor.

21 Así se puede dejar descargando cualquier distribución de *Linux*, por ejemplo.

22 Por ejemplo, el resultado de fecha y hora sería en griego:

```
Τετ 14 Δεκ 2016 07:16:20 μμ CET
```

En español sería:

```
mié dic 14 19:18:23 CET 2016
```

23 El mensaje de información sobre la versión es el siguiente:

```
date (GNU coreutils) 8.25
Copyright (C) 2016 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later
<http://gnu.org/licenses/gpl.html>.
This is free software: you are free to change and
redistribute it.
There is NO WARRANTY, to the extent permitted by law.

Written by David MacKenzie.
```

Habla del número de versión, la licencia de uso y de quién ha escrito el programa. Ésta es la versión en inglés.

24 O habría que pronunciarla con acento —entonación de voz— en la *i*.

25 Por supuesto, puede haber fallos debidos a equivocaciones de la persona que da las órdenes —errores de programación—. Sin embargo, la máquina no puede equivocarse, sólo calcula automáticamente.

26 Si lo fueran para un robot, no dejarían de ser herramientas o instrumentos para ese robot.

27 En realidad, fue su hermano Josef quien se la sugirió.

28 En puridad, «androide» es «con apariencia de varón». «Con aspecto humano» es «antropoide» —de ἀνθρωποειδής—, si bien ese adjetivo tiene un significado muy concreto en biología. «Con apariencia de mujer» sería «ginecoide» —de γυναικοειδής—

29 Propiamente no entiende, sino que es capaz de procesar determinados mensajes que contienen órdenes.

30 Se encuentra expuesto en C.E. Shannon, *A Mathematical Theory of Communication*.

31 Y ginecoides, o antropoides electrónicos.

32 Ese funcionamiento mágico también puede fallar estrepitosamente, como cuando un tipo de archivo está vinculado de modo predeterminado a un programa que no le corresponde.

33 Como es obvio, un archivo de sonido no admite impresión.

34 El ordenador en que escribo tiene *Fedora 25 LXDE*, que viene sin *LibreOffice* incluido en la instalación.

35 *Gnumeric*, en `http://www.gnumeric.org/`.

36 Si reasignamos la extensión `.pdf` a un programa de conversión de archivos, tendremos un problema para ver sin más los documentos PDF. Por poner un ejemplo entre otros muchos.

37 Queen, *I Want It All*, en *The Miracle*, vídeo oficial disponible en `https://www.youtube.com/watch?v=hFDcoX7s6rE`.

38 El caso más claro podría ser publicar en internet una nueva versión de un sitio completo de internet. Puede tener infinitas páginas, pero se trata de borrar lo antiguo, copiar lo nuevo y no tocar lo que no ha cambiado. Por si alguien duda, es una orden de una línea.

Este documento se ha generado con
`pandoc` (`http://pandoc.org/`).

La composición tipográfica del documento
PDF se ha realizado usando ConTEXt
(`http://contextgarden.net/`).

Las tipografías usadas para el documento PDF son *TEX Gyre Pagella*, *GFS Didot*, *URW Classico* y *Cousine*.
*Tempestiva* y *Cousine* se emplean en el documento ePub